



4b. Image Processing

Never Stand Still

Faculty of Engineering

COMP9444 Week 4b

Hao Xue

School of Computer Science and Engineering

Faculty of Engineering

The University of New South Wales, Sydney, Australia

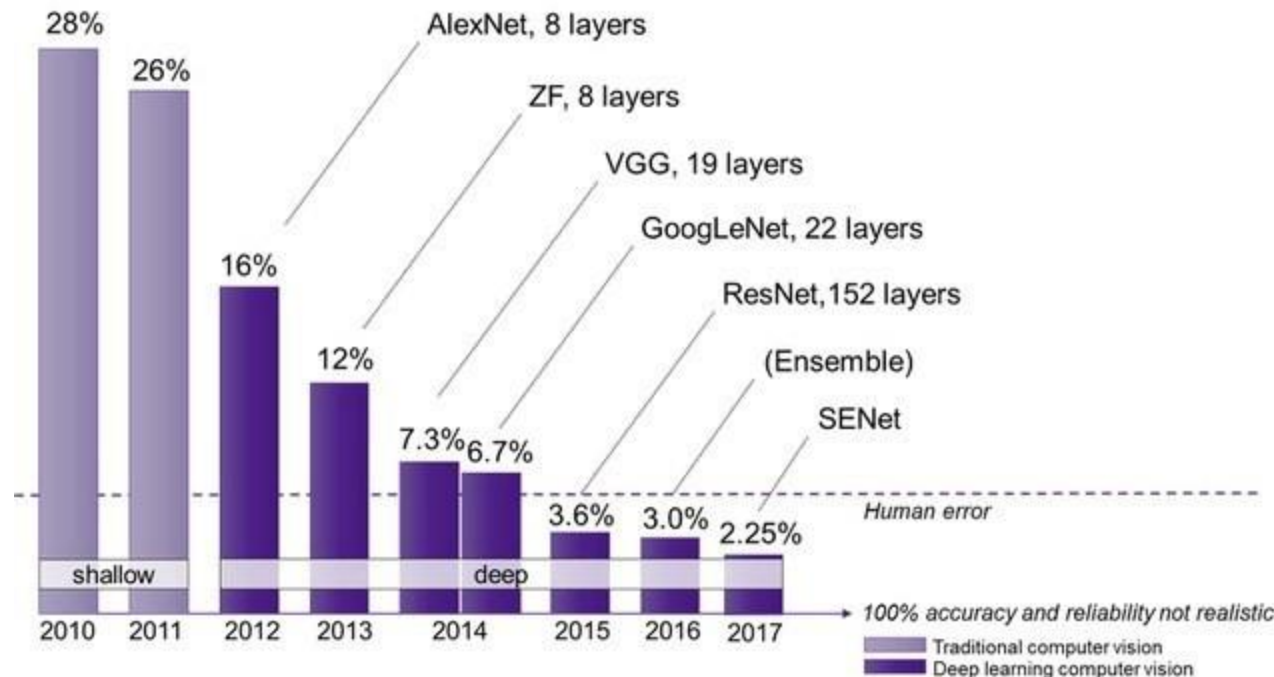
cs9444@cse.unsw.edu.au

Agenda

- Convolutional Neural Networks
- Why training Deep Neural Networks is hard?
- DNN training strategy
- Transfer Learning
- Overfitting and Underfitting
- Methods to avoid overfitting
 - Data Augmentation
 - Regularization
- Data Preprocessing
- Batch Normalization
- Choice of optimizers
- Tuning DNNs hyperparameters
- Evaluation

Convolutional Neural Networks (CNNs)

- A class of deep neural networks suitable for processing 2D/3D data. For e.g., Images and Videos
- CNNs can capture high-level representation of images/videos which can be used for end-tasks such as classification, object detection, segmentation, etc.
- A range of CNNs improving over the years



History

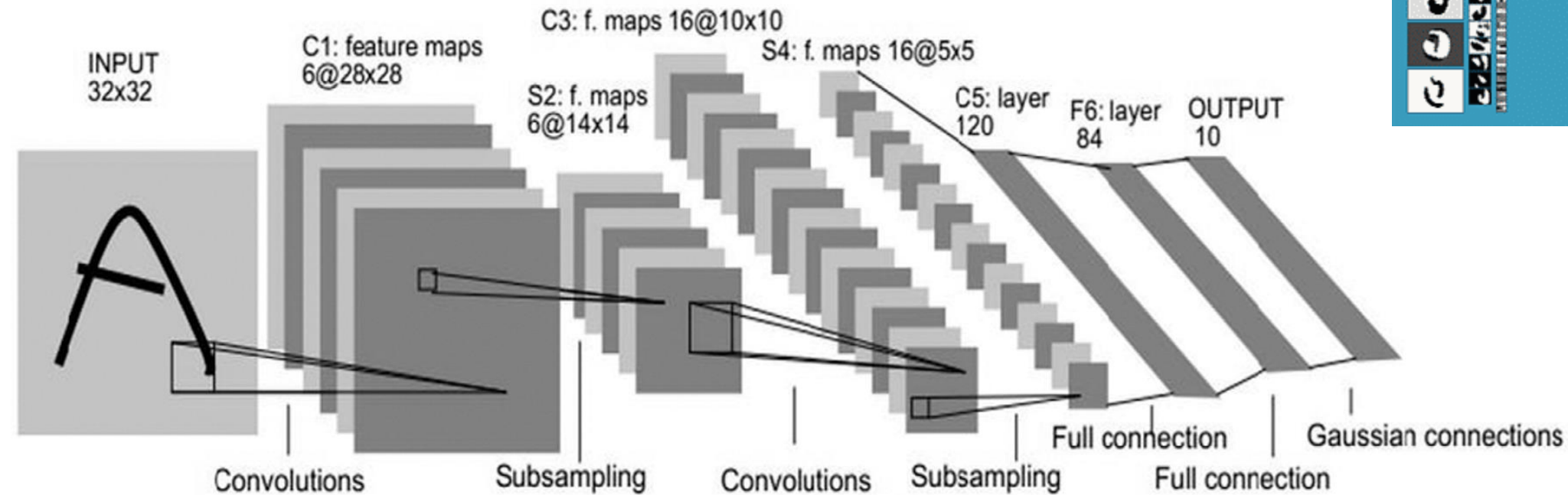
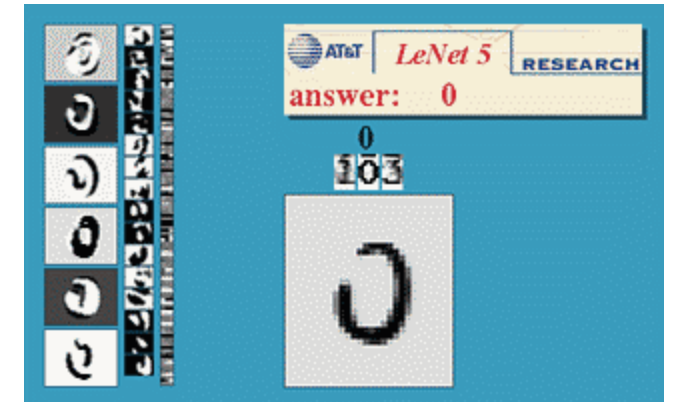
- ImageNet (2009)
 - Consists of 14 million images, more than 21,000 classes, and about 1 million images have bounding box annotations
 - Annotated by humans using crowdsourcing platform “Amazon Mechanical Turk”



- ImageNet Large-Scale Visual Recognition Challenge (ILSVRC)
 - annual competition to foster the development and benchmarking of state-of-the-art algorithms in Computer Vision
 - Led to improvement in architectures and techniques at the intersection of CV and DL

LeNet

- First developed by Yann Lecun in 1989 for digit recognition
 - First time backprop is used to automatically learn visual features
 - Two convolutional layers, three fully connected layers (32 x 32 input, 6 and 12 FMs, 5 x 5 filters)
 - Stride = 2 is used to reduce image dimensions
 - Scaled tanh activation function
 - Uniform random weight initialization

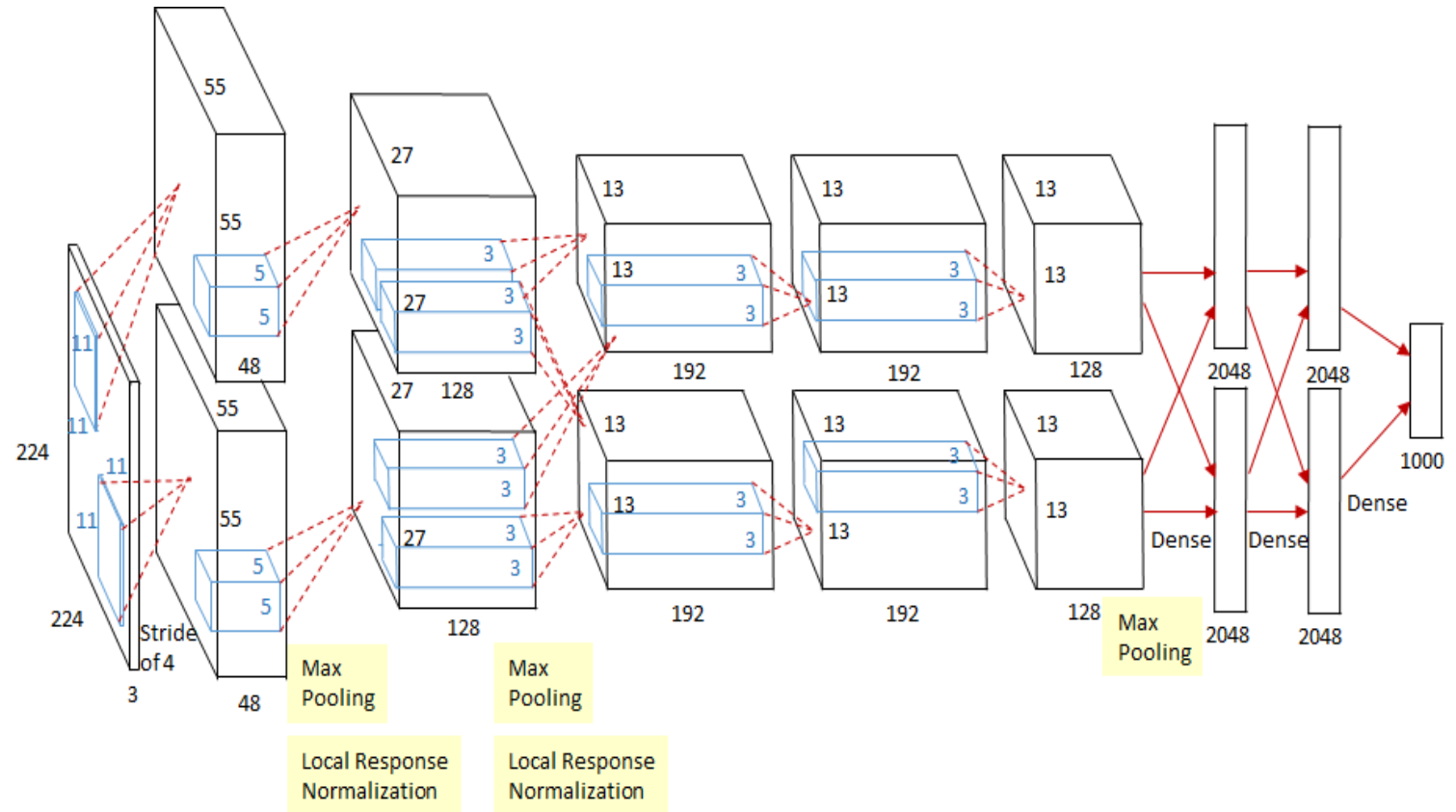


CNN Architectures

- AlexNet, 8 layers (2012)
- VGG, 19 layers (2014)
- GoogleNet, 22 layers (2014)
- ResNets, 152 layers (2015)
- DenseNet, 201 layers (2017)
- EfficientNet (2019)
- EfficientNetV2 (2021)

AlexNet

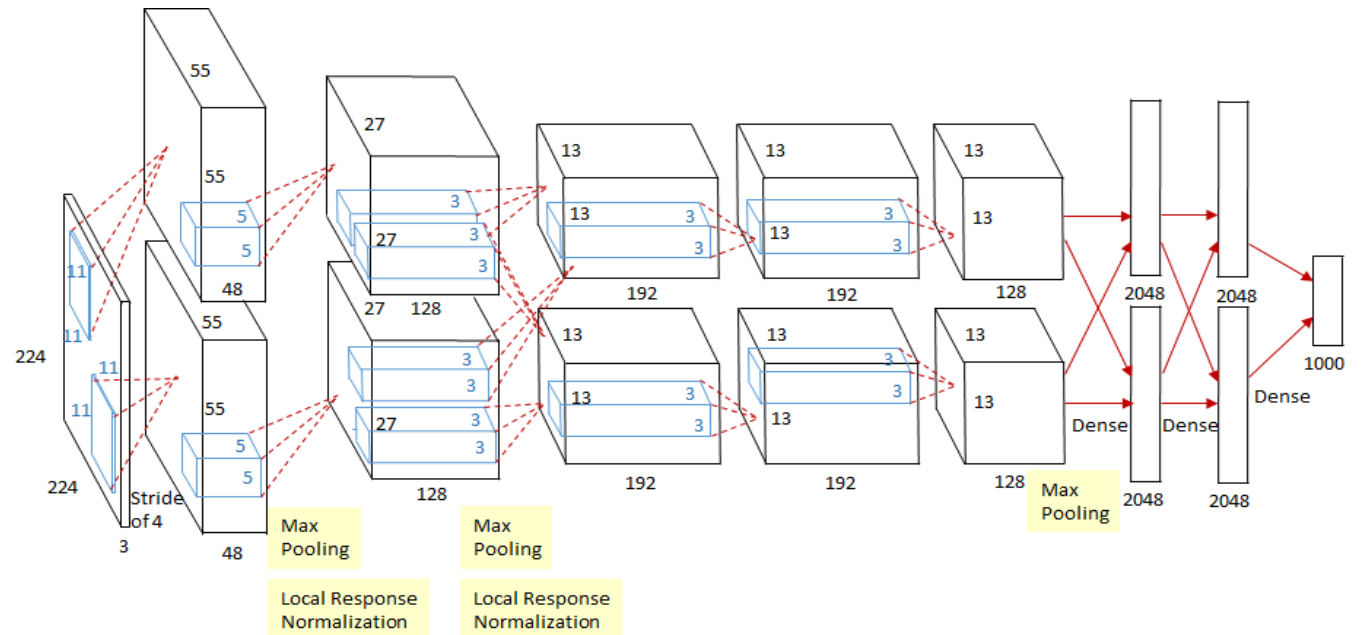
- 650K neurons
- 630M connections
- 60M parameters



- more parameters than images -----> danger of overfitting

Enhancements

- Rectified Linear Units (ReLU)s
- Overlapping pooling (Width = 3, stride = 2)
- Stochastic gradient descent with momentum and weight decay
- Data augmentation to reduce overfitting
- 50% dropout in the fully connected layers



Dealing with Deep Networks

➤ > 10 layers

- weight initialization
- batch normalization

➤ > 30 layers

- skip connections

➤ > 100 layers

- identity skip connections

Statistics Example: Coin Tossing

Example: Toss a coin once, and count the number of Heads

$$\text{Mean } \mu = \frac{1}{2} (0 + 1) = 0.5$$

$$\text{Variance} = \frac{1}{2} ((0 - 0.5)^2 + (1 - 0.5)^2) = 0.25$$

$$\text{Standard Deviation } \sigma = \sqrt{\text{Variance}} = 0.5$$

Example: Toss a coin 100 times, and count the number of Heads

$$\text{Mean } \mu = 100 * 0.5 = 50$$

$$\text{Variance} = 100 * 0.25 = 25$$

$$\text{Standard Deviation } \sigma = \sqrt{\text{Variance}} = 5$$

Example: Toss a coin 10000 times, and count the number of Heads

$$\mu = 5000, \quad \sigma = \sqrt{2500} = 50$$

Statistics

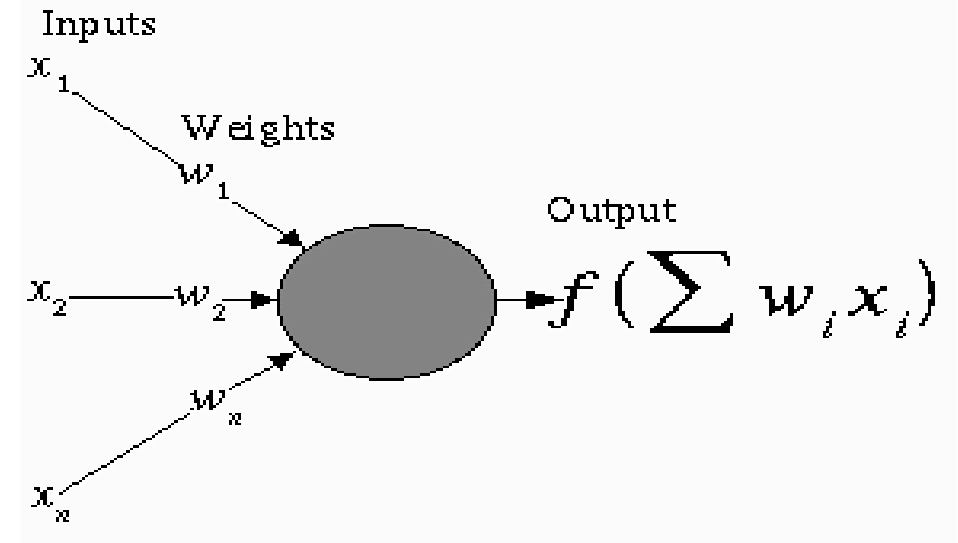
The mean and variance of a set of n samples $x_1, \dots, x_n$ are given by

$$\text{Mean}[x] = \frac{1}{n} \sum_{k=1}^n x_k$$

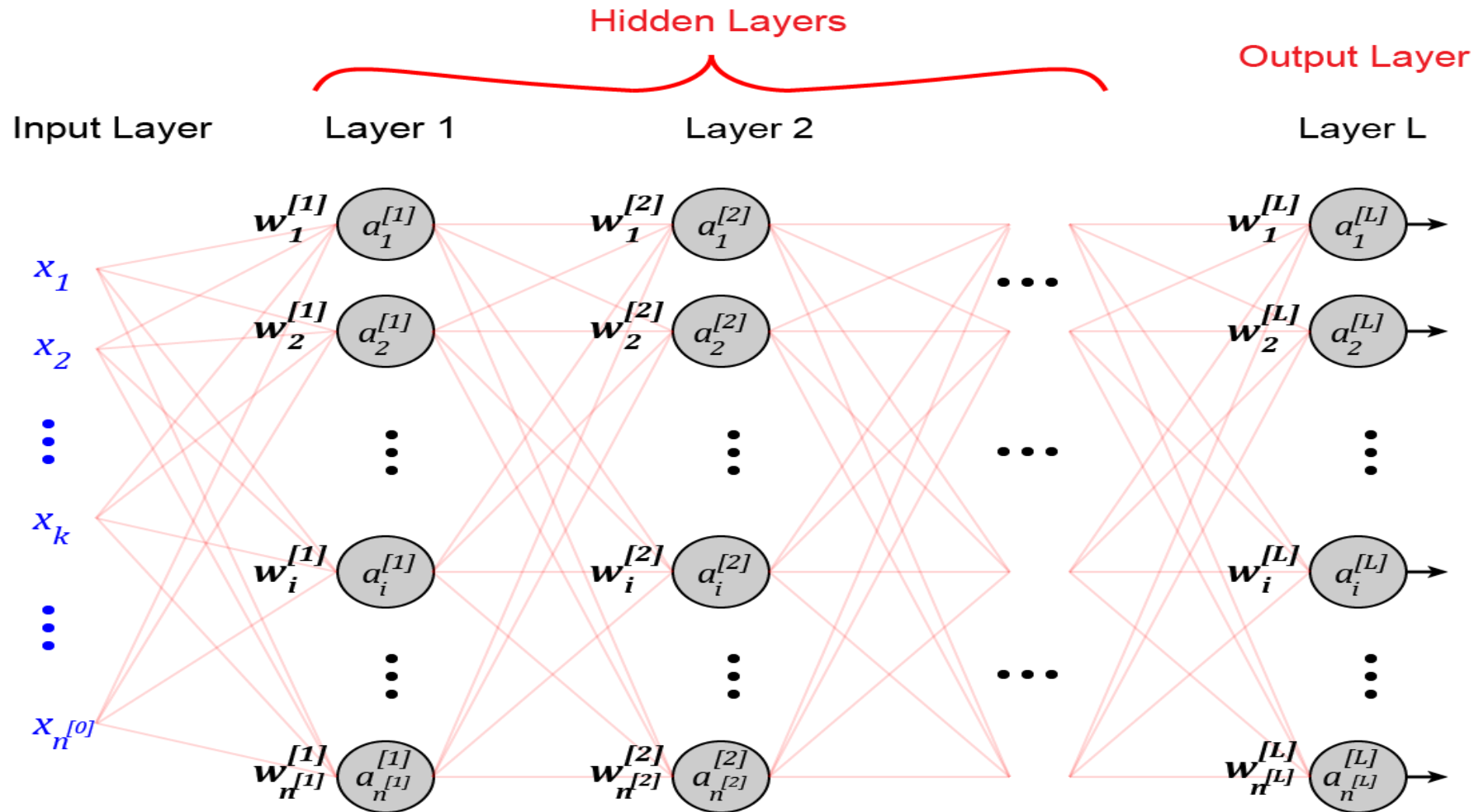
$$\text{Var}[x] = \frac{1}{n} \sum_{k=1}^n (x_k - \text{Mean}[x])^2 = \left(\frac{1}{n} \sum_{k=1}^n x_k^2 \right) - \text{Mean}[x]^2$$

If w_k, x_k are independent and $y = \sum_{k=1}^n w_k x_k$ then

$$\text{Var}[y] = n \text{Var}[w] \text{Var}[x]$$



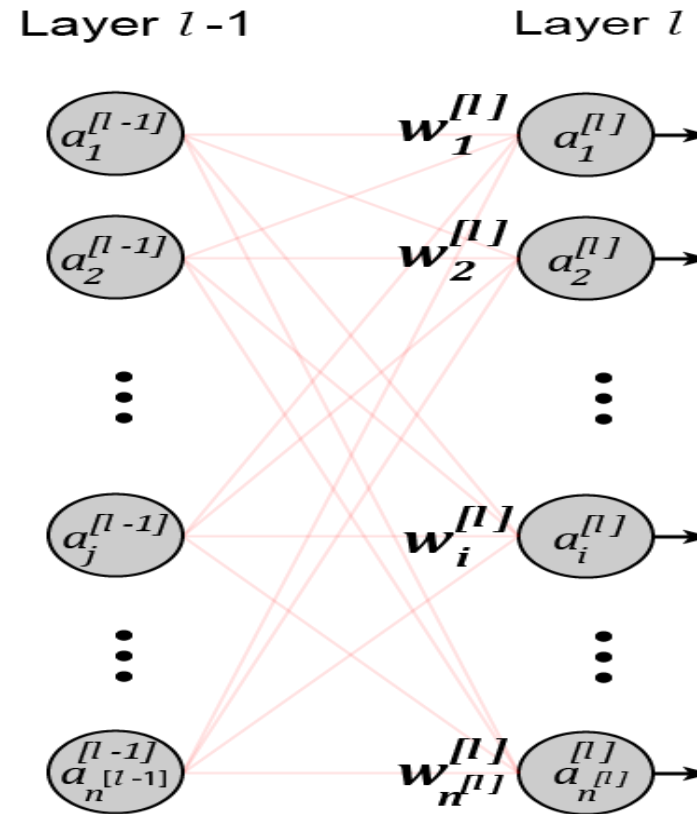
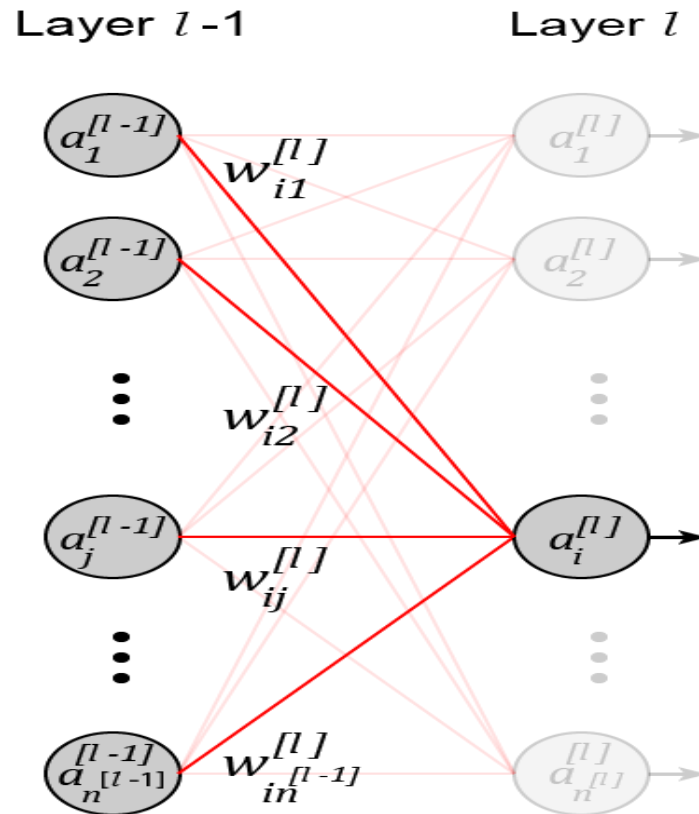
Weight Initialization



Weight Initialization

Consider one layer (i) of a deep neural network with weights $w_{jk}^{(i)}$ connecting the activations $\{x_k^{(i)}\}_{1 \leq k \leq n_i}$ at the previous layer to $\{x_j^{(i+1)}\}_{1 \leq j \leq n_{i+1}}$ at the next layer, where $g(\cdot)$ is the transfer function and

$$x_j^{(i+1)} = g(\text{sum}_j^{(i)}) = g\left(\sum_{k=1}^{n_i} w_{jk}^{(i)} x_k^{(i)}\right)$$



Weight Initialization

Consider one layer (i) of a deep neural network with weights $w_{jk}^{(i)}$ connecting the activations $\{x_k^{(i)}\}_{1 \leq k \leq n_i}$ at the previous layer to $\{x_j^{(i+1)}\}_{1 \leq j \leq n_{i+1}}$ at the next layer, where $g(\cdot)$ is the transfer function and

$$x_j^{(i+1)} = g(\text{sum}_j^{(i)}) = g\left(\sum_{k=1}^{n_i} w_{jk}^{(i)} x_k^{(i)}\right)$$

Then

$$\text{Var}[\text{sum}^{(i)}] = n_i \text{Var}[w^{(i)}] \text{Var}[x^{(i)}]$$

$$\text{Var}[x^{(i+1)}] \simeq G_0 n_i \text{Var}[w^{(i)}] \text{Var}[x^{(i)}]$$

Where G_0 is a constant whose value is estimated to take account of the transfer function.

If some layers are not fully connected, we replace n_i with the average number n_i^{in} of incoming connections to each node at layer $i + 1$.

Weight Initialization

If the network has D layers, with input $x = x^{(1)}$ and output $z = x^{(D+1)}$, then

$$\text{Var}[z] \simeq \left(\prod_{i=1}^D G_0 n_i^{\text{in}} \text{Var}[w^{(i)}] \right) \text{Var}[x]$$

When we apply gradient descent through backpropagation, the differentials will follow a similar pattern:

$$\text{Var}\left[\frac{\partial}{\partial x}\right] \simeq \left(\prod_{i=1}^D G_1 n_i^{\text{out}} \text{Var}[w^{(i)}] \right) \text{Var}\left[\frac{\partial}{\partial z}\right]$$

In this equation, n_i^{out} is the average number of outgoing connections for each node at layer i , and G_1 is meant to estimate the average value of the derivative of the transfer function.

For Rectified Linear Units, we can assume $G_0 = G_1 = \frac{1}{2}$

Weight Initialization

In order to have healthy forward and backward propagation, each term in the product must be approximately equal to 1. Any deviation from this could cause the activations to either vanish or saturate, and the differentials to either decay or explode exponentially.

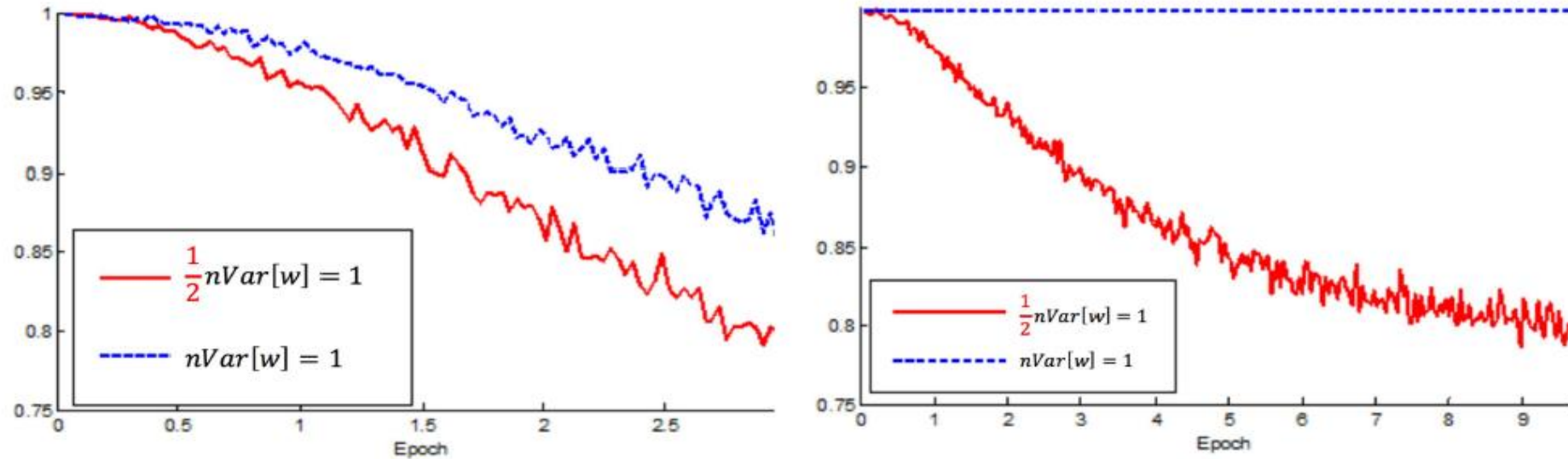
$$\text{Var}[z] \simeq \left(\prod_{i=1}^D G_0 n_i^{\text{in}} \text{Var}[w^{(i)}] \right) \text{Var}[x]$$

$$\text{Var}\left[\frac{\partial}{\partial x}\right] \simeq \left(\prod_{i=1}^D G_1 n_i^{\text{out}} \text{Var}[w^{(i)}] \right) \text{Var}\left[\frac{\partial}{\partial z}\right]$$

We therefore choose the initial weights $\{w_{jk}^{(i)}\}$ in each layer (i) such that

$$G_1 n_i^{\text{out}} \text{Var}[w^{(i)}] = 1$$

Weight Initialization



- 22-layer ReLU network (left),
 $\text{Var}[w] = \frac{2}{n}$ converges faster than $\text{Var}[w] = \frac{1}{n}$
- 30-layer ReLU network (right),
 $\text{Var}[w] = \frac{2}{n}$ is successful while $\text{Var}[w] = \frac{1}{n}$ fails to learn at all

Batch Normalization

We can **normalize** the activations $x_k^{(i)}$ of node k in layer (i) relative to the mean and variance of those activations, calculated over a mini-batch of training items:

$$\hat{x}_k^{(i)} = \frac{x_k^{(i)} - \text{Mean}[x_k^{(i)}]}{\sqrt{\text{Var}[x_k^{(i)}]}}$$

These activations can then be shifted and re-scaled to

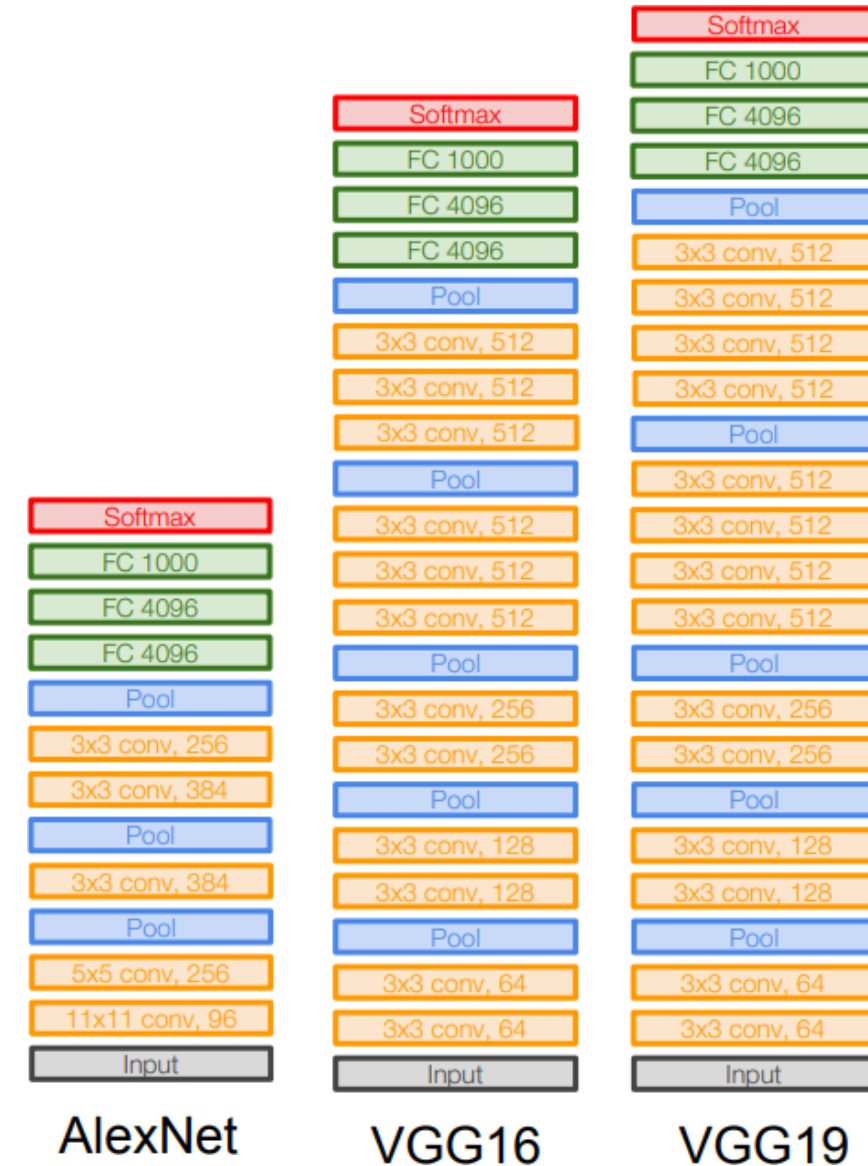
$$y_k^{(i)} = \beta_k^{(i)} + \gamma_k^{(i)} \hat{x}_k^{(i)}$$

$\beta_k^{(i)}, \gamma_k^{(i)}$ are additional parameters, for each node, which are trained by backpropagation along with the other parameters (weights) in the network.

After training is complete, $\text{Mean}[x_k^{(i)}]$ and $\text{Var}[x_k^{(i)}]$ are either pre-computed on the entire training set, or updated using running averages.

VGG

- Developed at Visual Geometry Group (Oxford) by Simonyan and Zisserman
- 1st runner up (Classification) and Winner (localization) of ILSVRC 2014 competition
- VGG-19 comprises of 144 million parameters
- Use smaller 3x3 convolutional kernels linked in series to achieve a larger receptive field
- Abandon the use of large-sized convolutional kernels such as 11x11 and 5x5
- Greater depth, stronger nonlinearity, and fewer network parameters
- Removed the Local Response Normalization (LRN) layer from AlexNet.

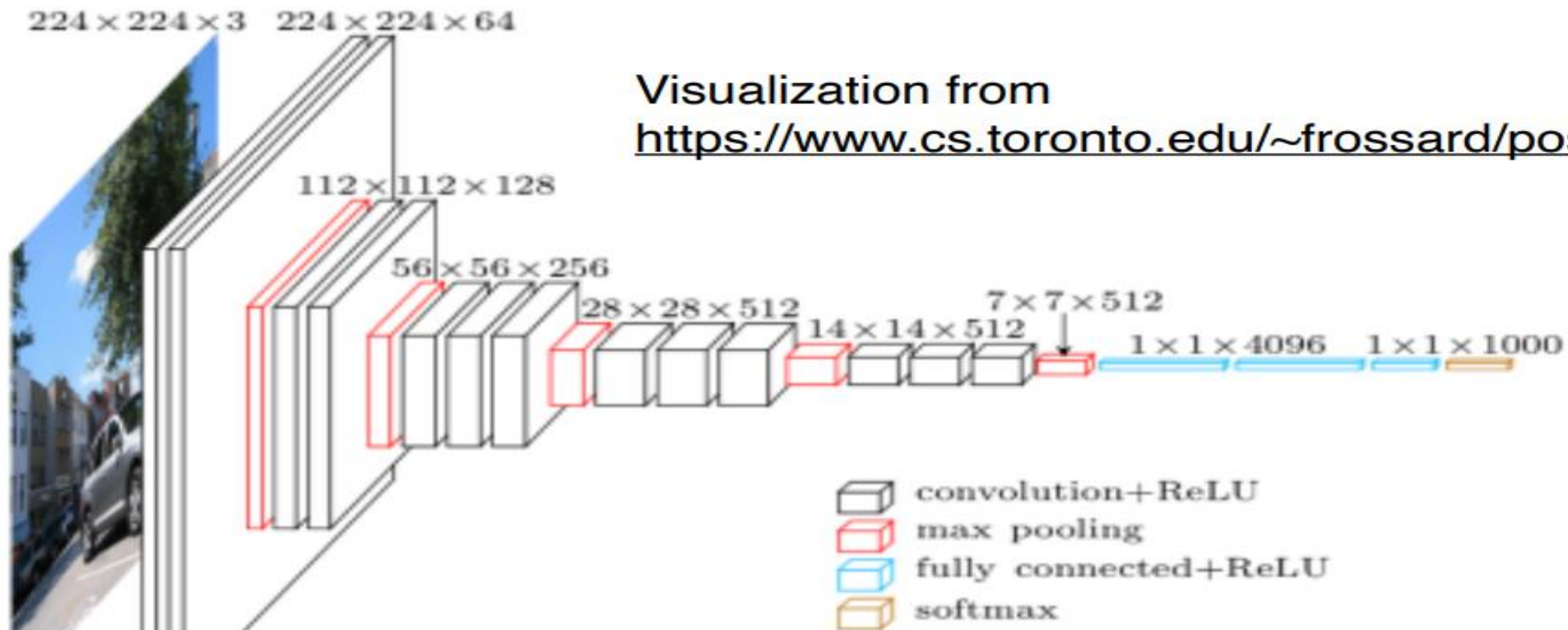


AlexNet

VGG16

VGG19

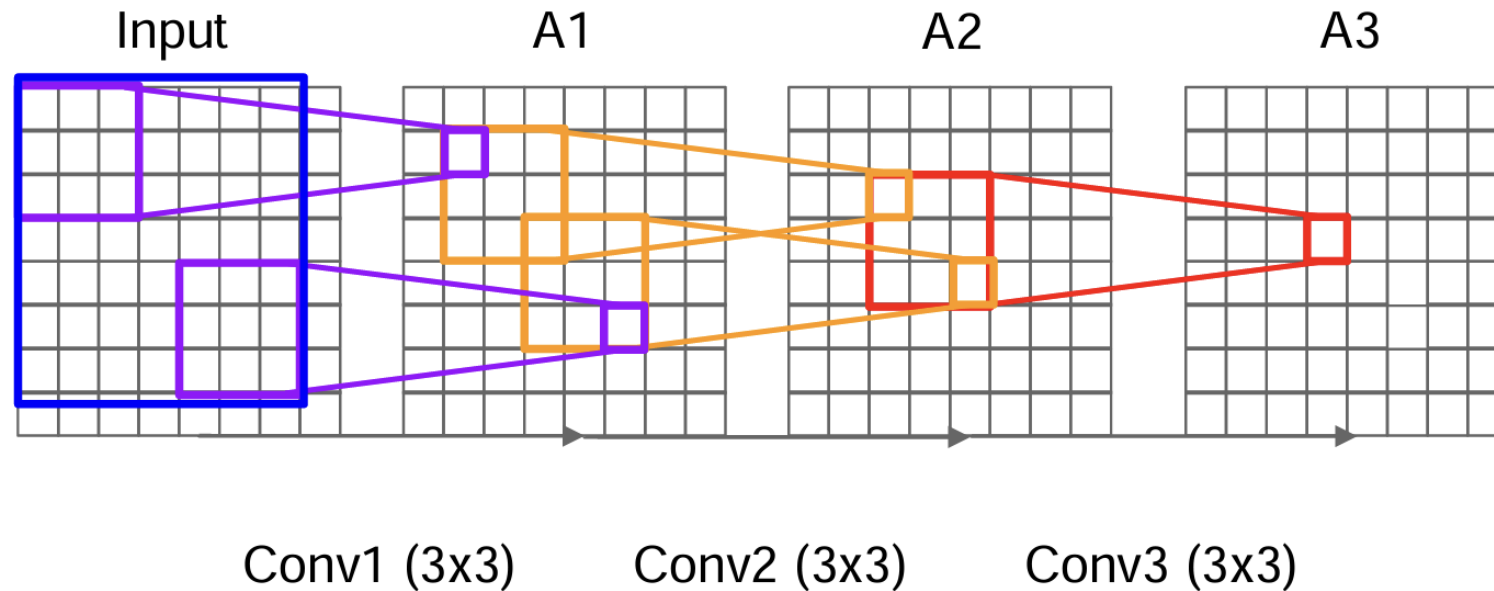
VGG-16



Simonyan, Karen, and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition." arXiv preprint arXiv:1409.1556 (2014).

Smaller Kernels?

Stack of three 3x3 conv (stride 1) layers has same effective receptive field as one 7x7 conv layer



Smaller Kernels?

But deeper, more non-linearities and fewer parameters

one 7x7 conv layer, C channels per layer

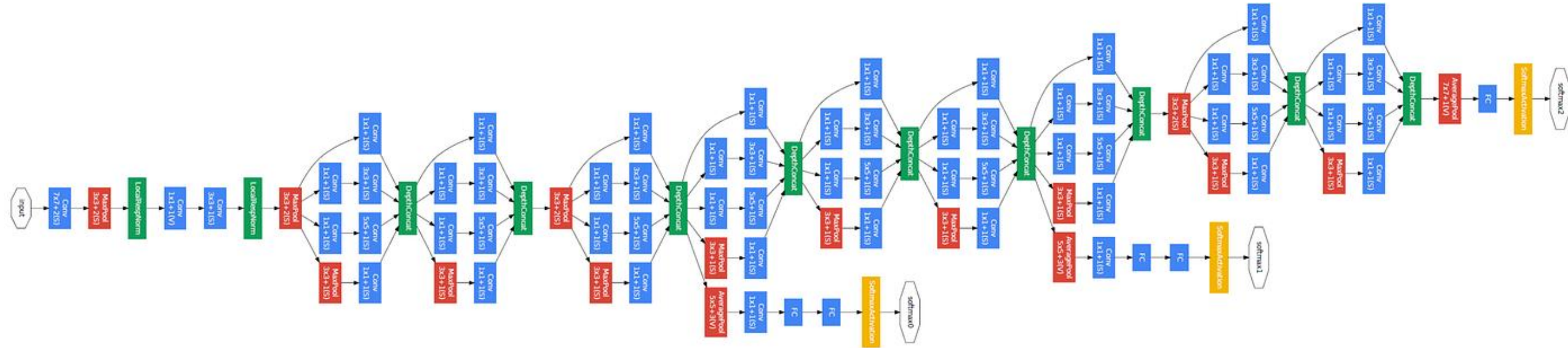
$$(7 \times 7 \times C) \times C = 49C^2$$

three 7x7 conv layers, C channels per layer

$$(3 \times 3 \times C) \times C \times 3 = 27C^2$$

GoogLeNet

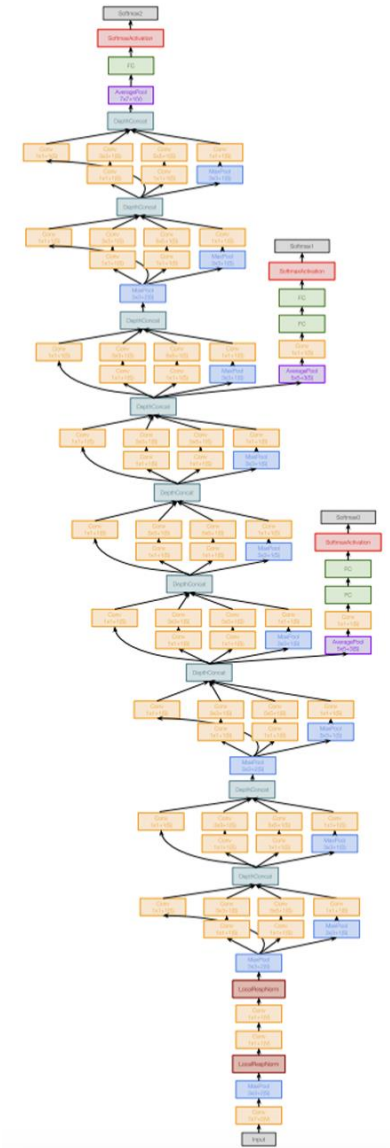
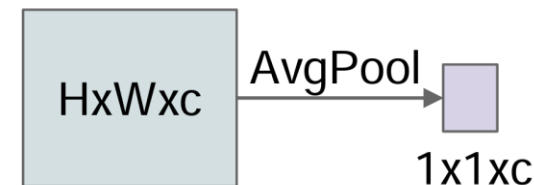
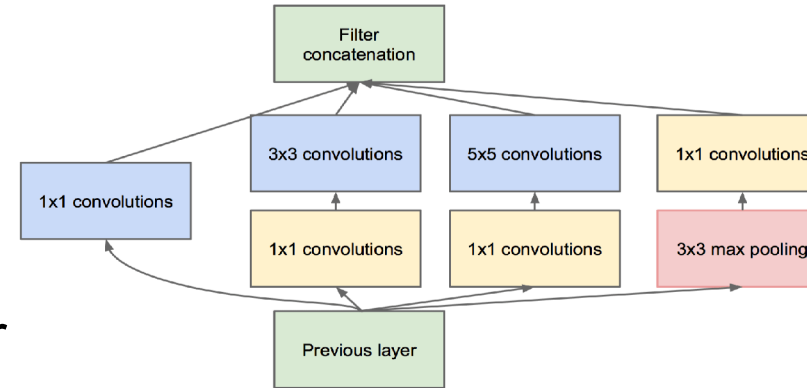
- Winner of 2014 ILSVRC Challenge
- A 22-layer CNN developed by researchers at Google



GoogLeNet

➤ Core idea “Inception module”

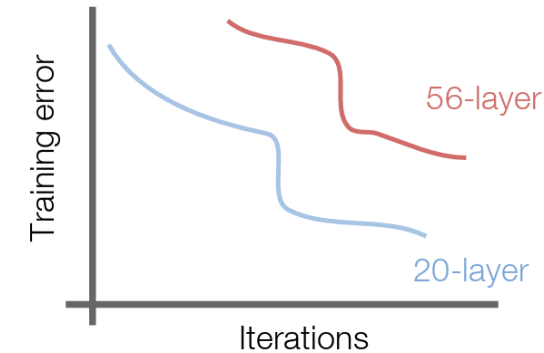
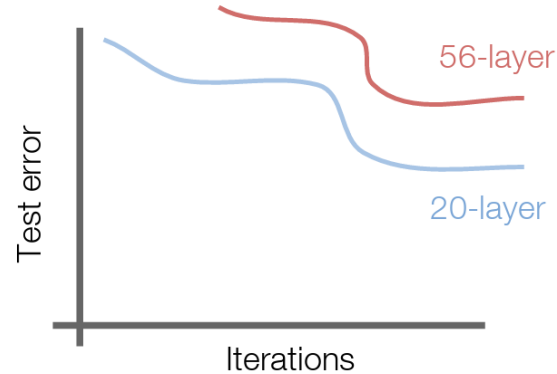
- Apply parallel filter operations on the input from previous layer
 - Concatenate all filter outputs together channel-wise
 - Multi-branch, Multi-size kernel
- Deeper networks prone to overfitting and suffer from exploding or vanishing gradient problem
- Adding Auxiliary loss as an extra supervision
- Global average pooling layer is used that spatially averages across each feature map, before final FC layer
- Reduce parameters!



Source: Convolutional Neural Networks. <https://medium.com/@rajat.k.91/convolutional-neural-networks-why-what-and-how-f8f6dbebb2f9>

ResNet

- Developed by researchers at Microsoft (Kaiming et al.)
- What happens when we continue stacking deeper layers on a “plain” convolutional neural network?

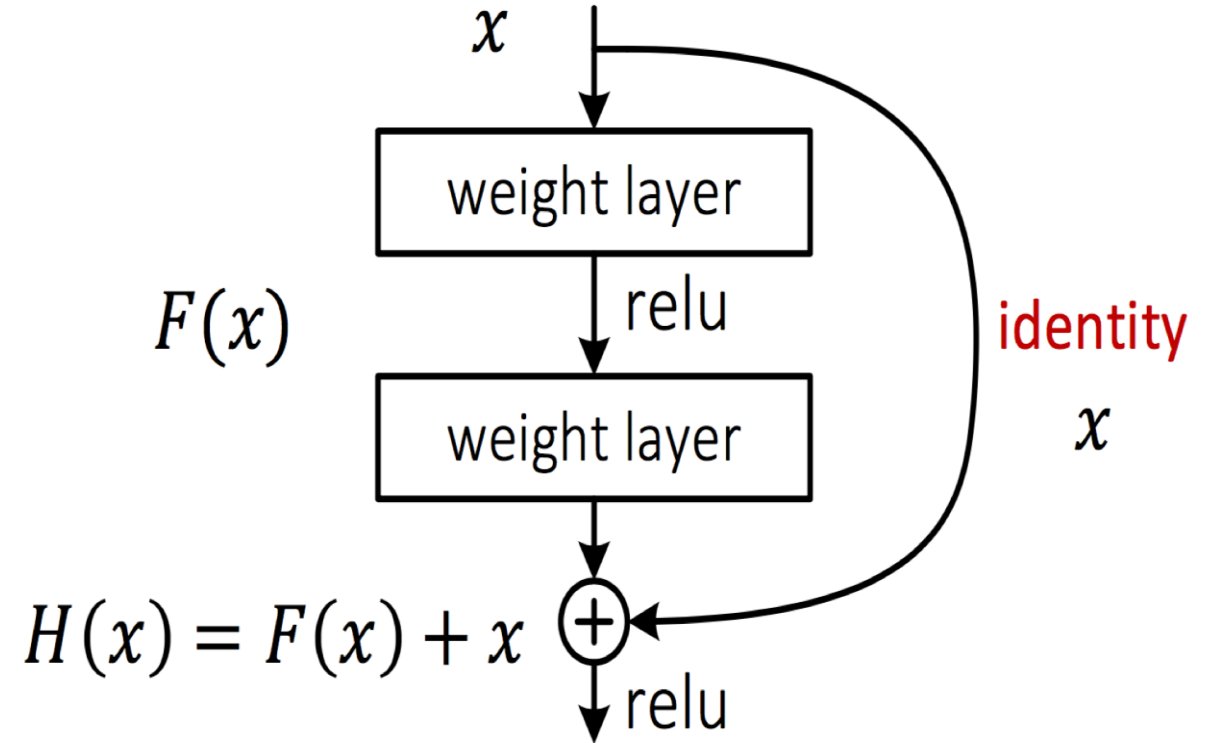
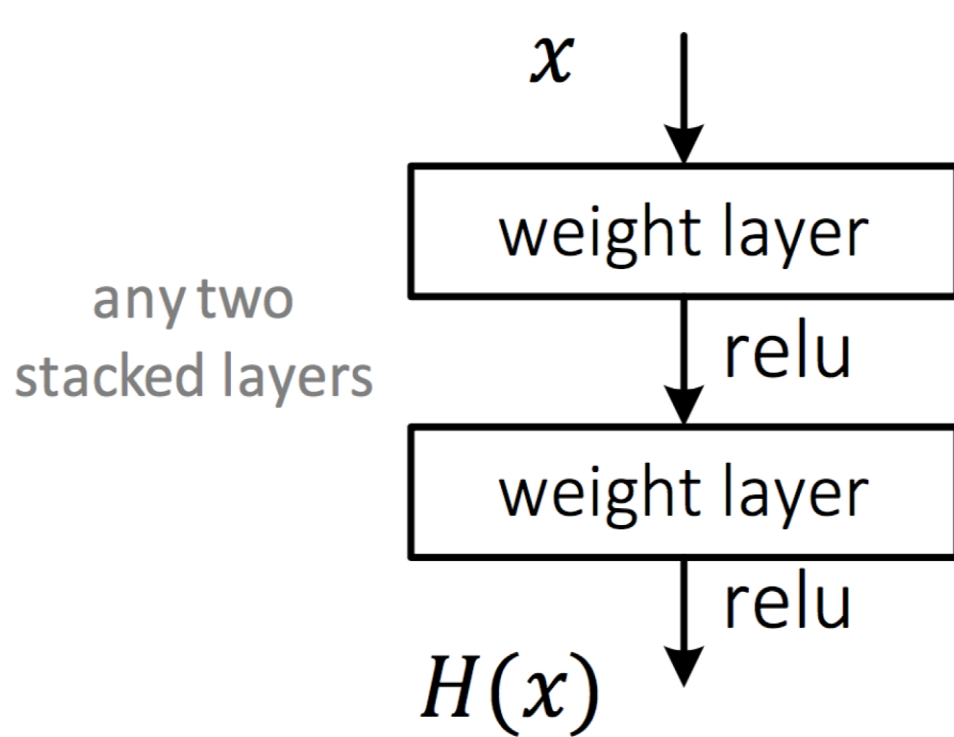


56-layer model performs worse on both test and training error

The deeper model performs worse, but it's not caused by overfitting!

Image Credit: Medium. https://medium.com/@pierre_guillou/understand-how-works-resnet-without-talking-about-residual-64698f157e0c

Residual Networks



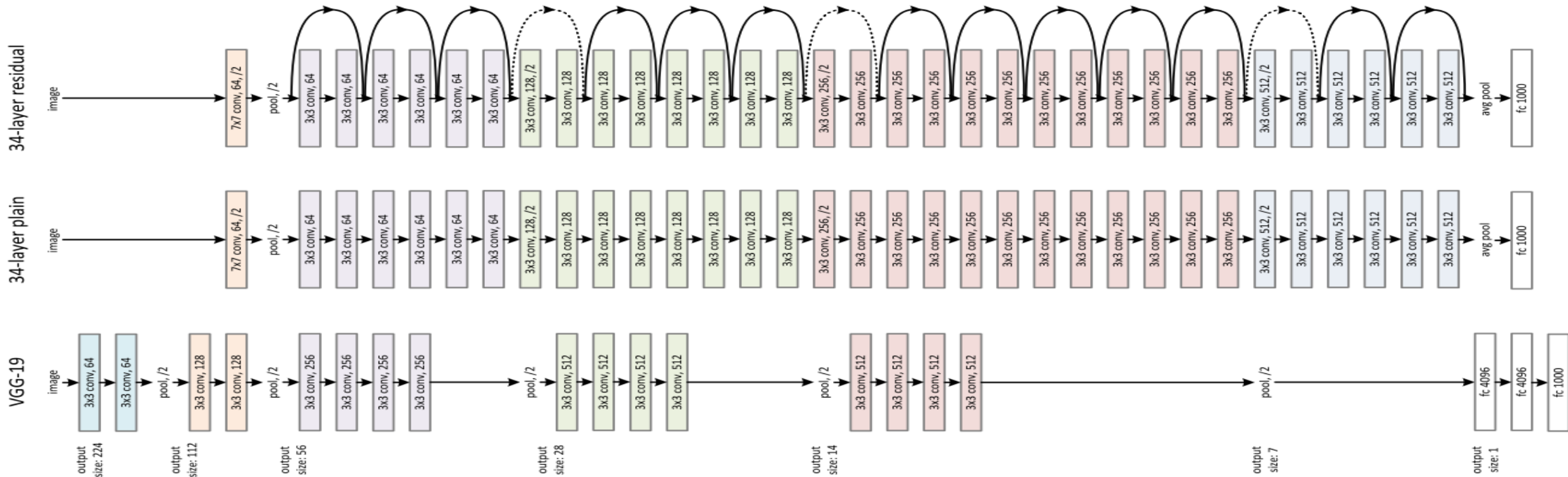
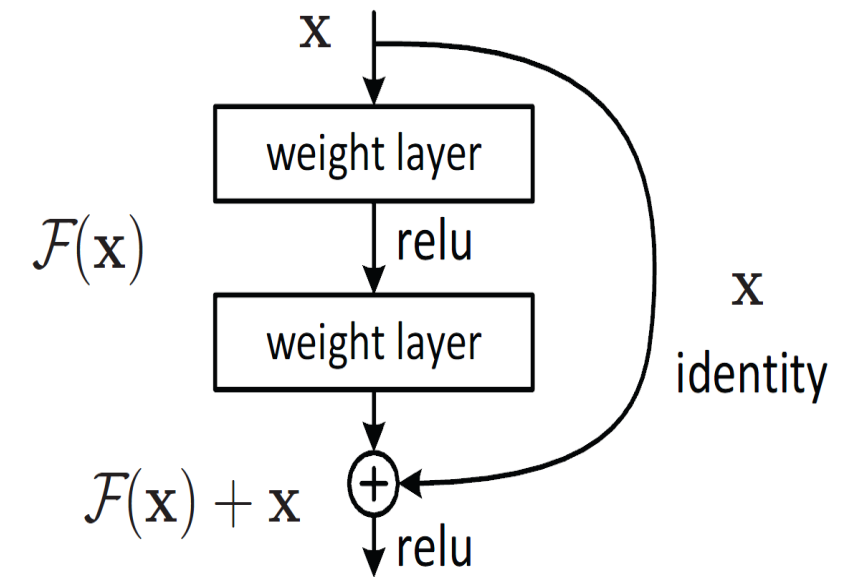
- **Idea:** Take any two consecutive stacked layers in a deep network and add a “skip” connection which bypasses these layers and is added to their output.

Residual Networks

- the preceding layers attempt to do the “whole” job, making x as close as possible to the target output of the entire network
- $F(x)$ is a residual component which corrects the errors from previous layers, or provides additional details which the previous layers were not powerful enough to compute
- With skip connections, both training and test error drop as you add more layers
- With more than 100 layers, need to apply ReLU before adding the residual instead of afterwards. This is called an **identity skip connection**.

ResNet

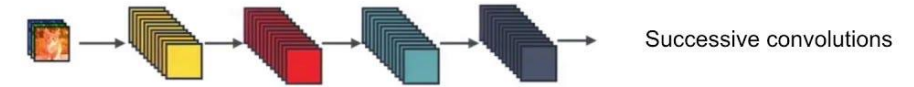
- Developed by researchers at Microsoft
- Core idea “**residual connections**” to preserve the gradient
- The identity matrix transmits forward the input data that avoids the loss of information (the data vanishing problem)



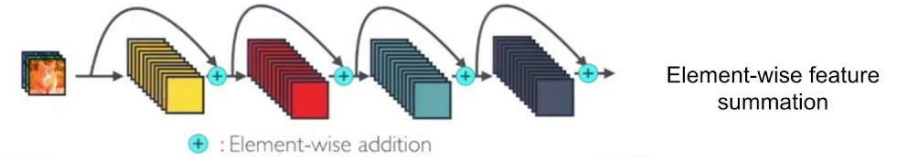
DenseNet

- “Dense block” vs “Res block”
- More flexible connections
- Transition layer is used between dense blocks
 - Reduce dimensionality and computation
 - 1x1 conv to reduce #feature maps
 - Avg pooling reduce spatial dimensions

Standard Connectivity



Resnet Connectivity



DenseNet Connectivity

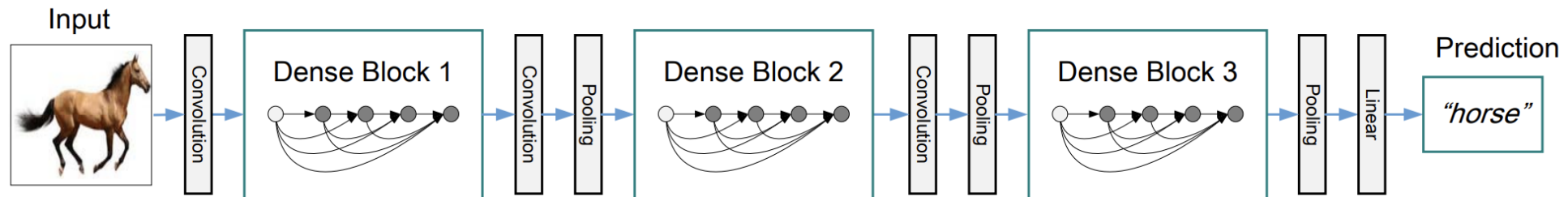
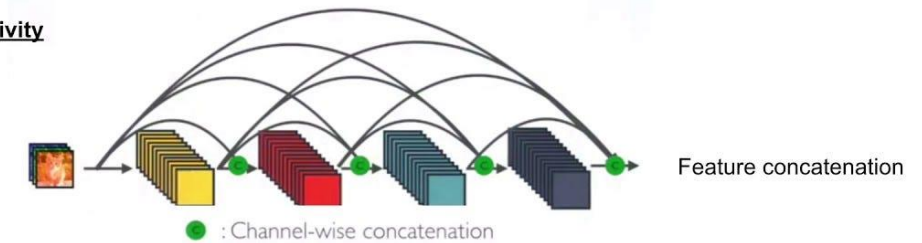
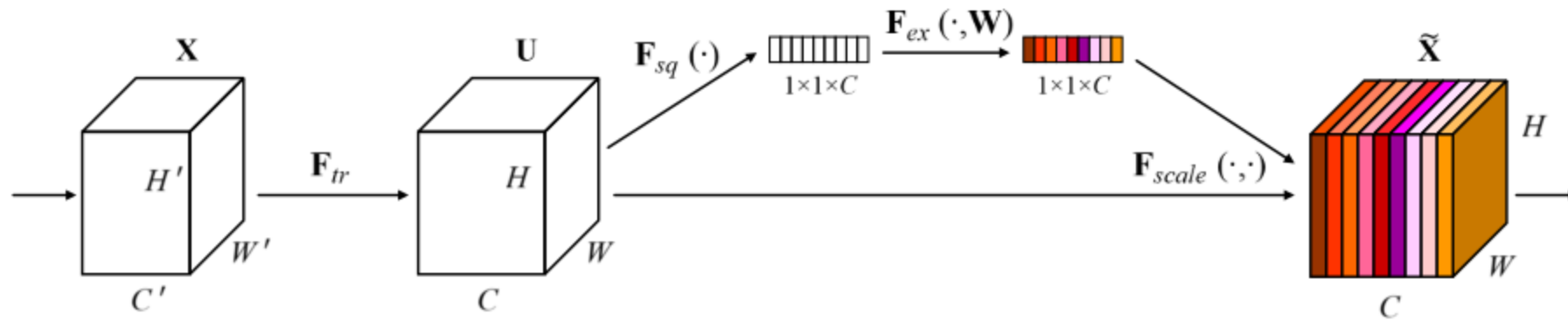


Figure 2: A deep DenseNet with three dense blocks. The layers between two adjacent blocks are referred to as transition layers and change feature-map sizes via convolution and pooling.

SENet (Squeeze-and-Excitation Network)

- CNNs fuse the spatial and channel information to extract features to solve the task
- Before this, networks weights each of its channels equally when creating the output feature maps
- SENets added a content aware mechanism to weight each channel adaptively
- SE block helps to improve representation power of the network, able to better map the channel dependency along with access to global information



Why training Deep Neural Networks is hard?



CNN: Training – PyTorch example

✓ Autograd in Training

We've had a brief look at how autograd works, but how does it look when it's used for its intended purpose? Let's examine how it changes after a single training batch. First, define a few constants, our model, and

```
[ ] BATCH_SIZE = 16
    DIM_IN = 1000
    HIDDEN_SIZE = 100
    DIM_OUT = 10

class TinyModel(torch.nn.Module):

    def __init__(self):
        super(TinyModel, self).__init__()

        self.layer1 = torch.nn.Linear(DIM_IN, HIDDEN_SIZE)
        self.relu = torch.nn.ReLU()
        self.layer2 = torch.nn.Linear(HIDDEN_SIZE, DIM_OUT)

    def forward(self, x):
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x

some_input = torch.randn(BATCH_SIZE, DIM_IN, requires_grad=False)
ideal_output = torch.randn(BATCH_SIZE, DIM_OUT, requires_grad=False)

model = TinyModel()
```

```
[ ] optimizer = torch.optim.SGD(model.parameters(), lr=0.001)

prediction = model(some_input)

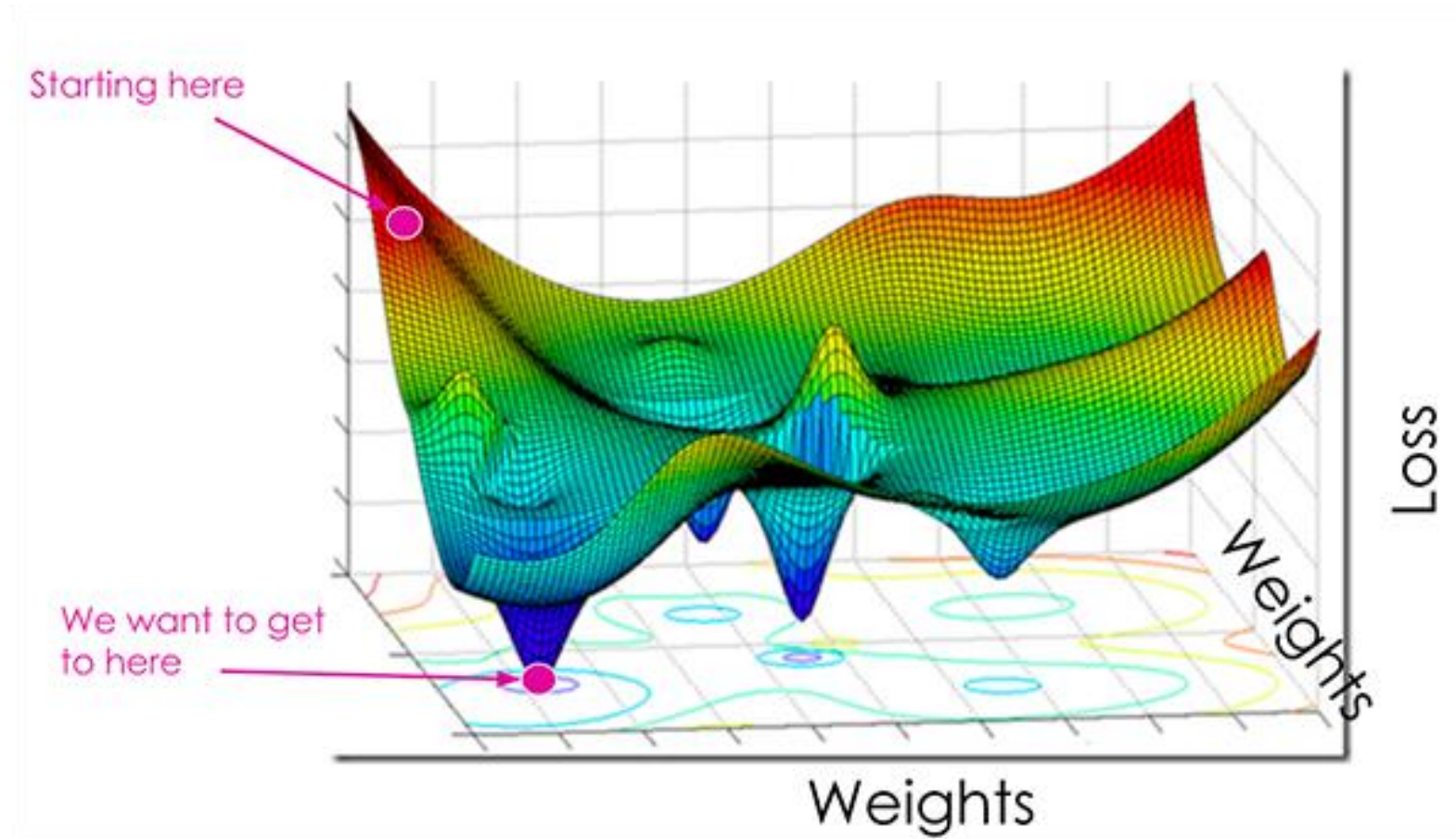
loss = (ideal_output - prediction).pow(2).sum()
print(loss)
```

```
[ ] loss.backward()
    print(model.layer2.weight[0][0:10])
    print(model.layer2.weight.grad[0][0:10])
```

```
[ ] optimizer.step()
```

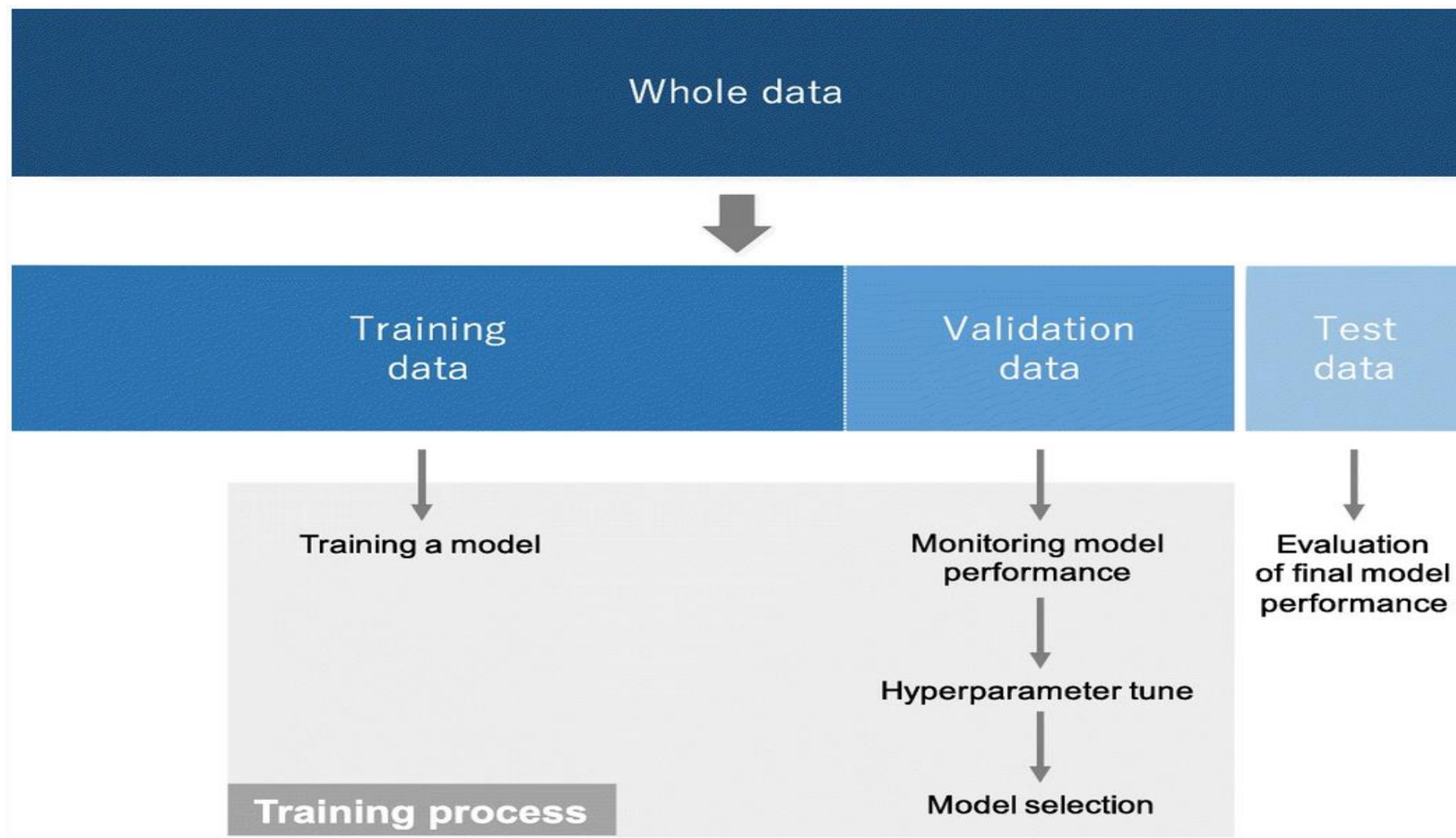
https://colab.research.google.com/github/pytorch/tutorials/blob/gh-pages/_downloads/ed9d4f94afb79f7dada6742a06c486a5/autogradyt_tutorial.ipynb

Why training Deep Neural Networks is hard?



Training Methodology

➤ steps

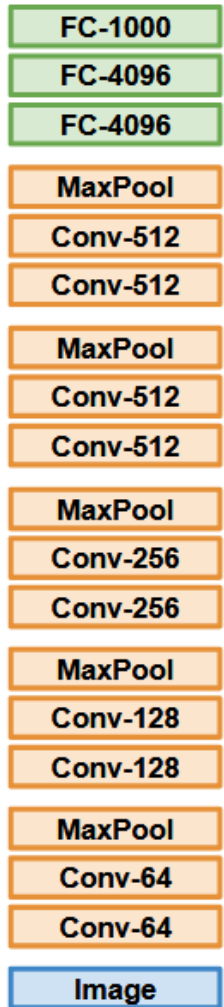


Transfer Learning

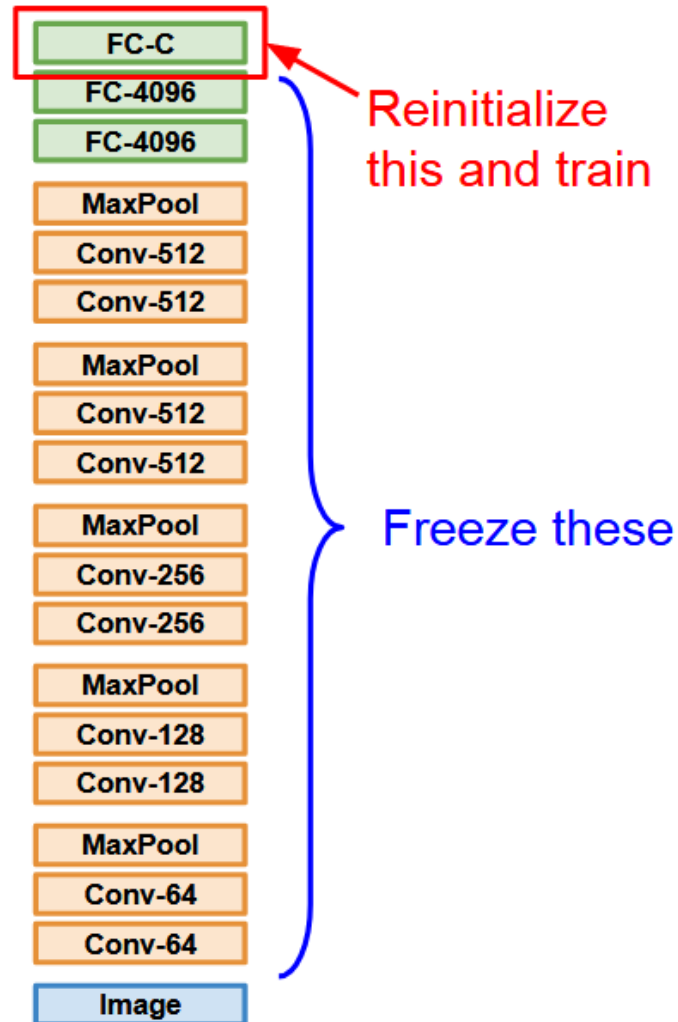
- Transfer learning aims to leverage the learned knowledge from a resource-rich domain/task to help learning a task with not sufficient training data.
 - Sometimes referred as **domain adaptation**
- The resource-rich domain is known as the **source** and the low-resource task is known as the **target**.
- Transfer learning works the best if the model features learned from the source task are **general** (i.e., domain-independent)

Transfer Learning with CNNs

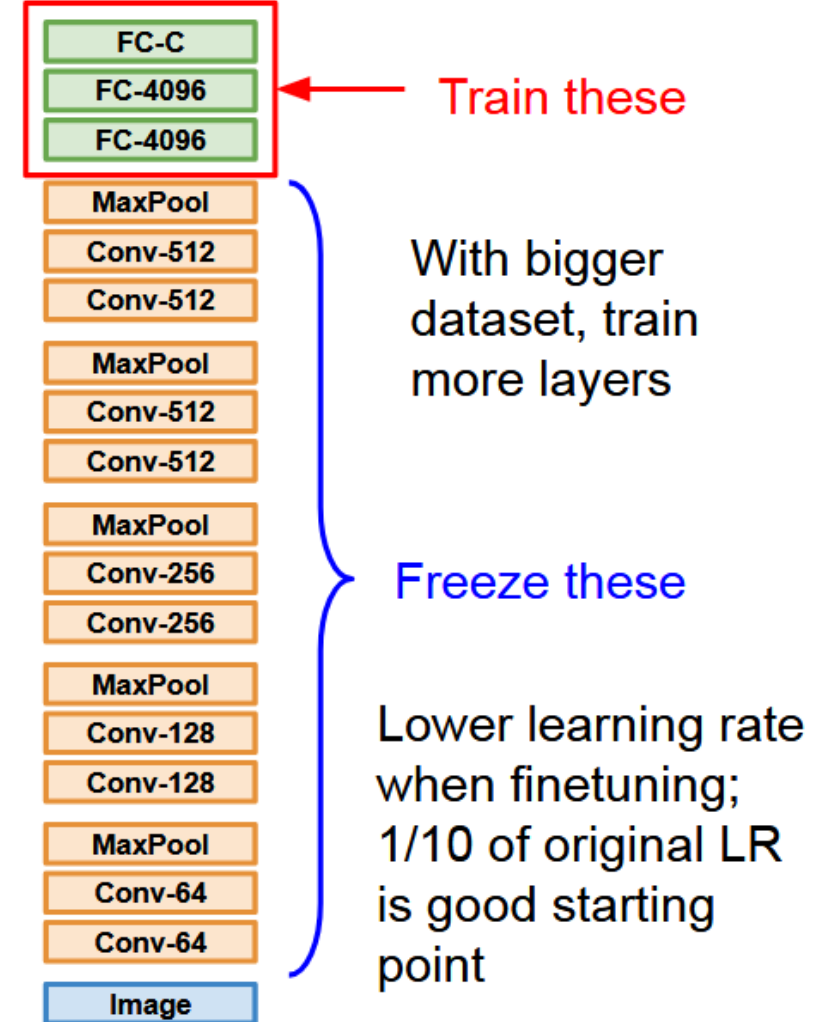
1. Train on Imagenet



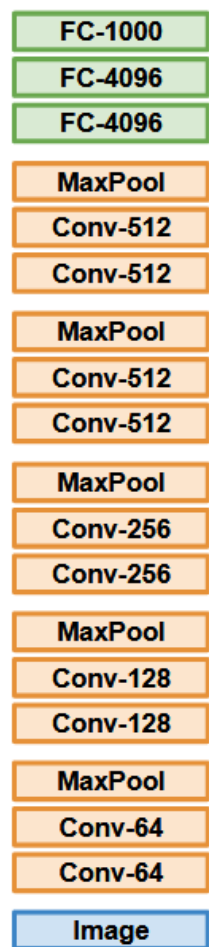
2. Small Dataset (C classes)



3. Bigger dataset



Transfer Learning with CNNs



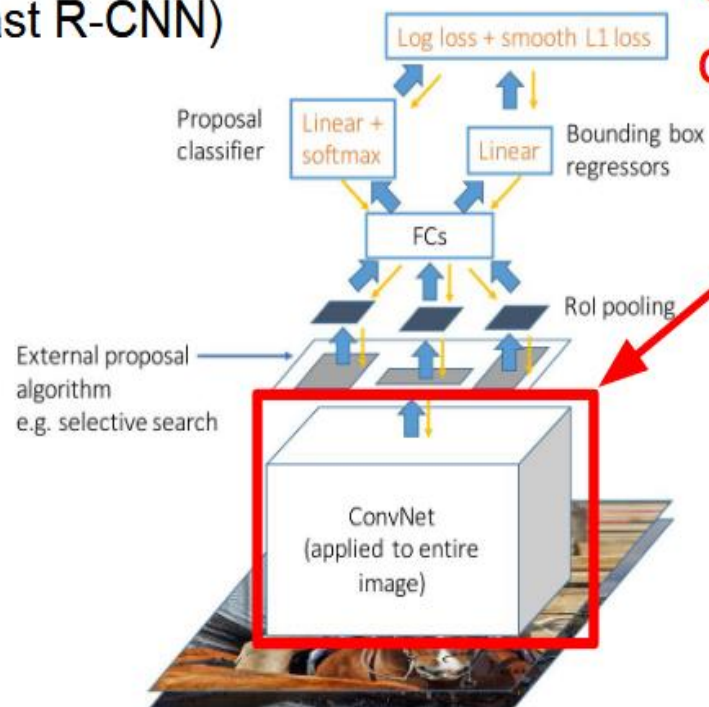
More specific

More generic

	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer	You're in trouble... Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

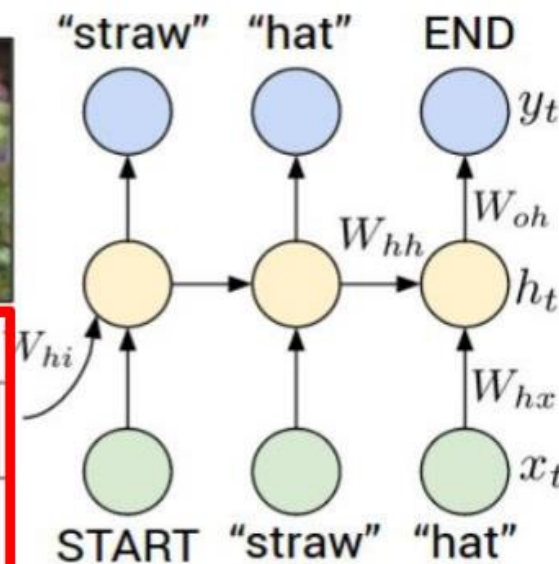
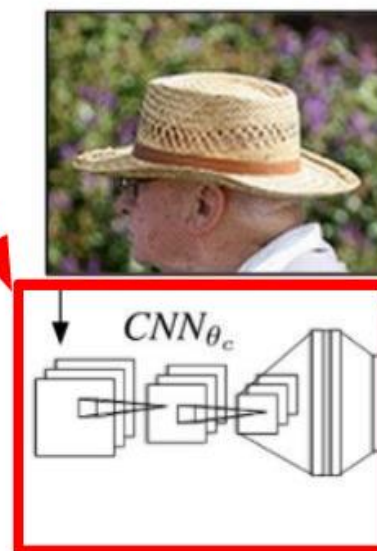
Transfer Learning is common in all applications

Object Detection (Fast R-CNN)



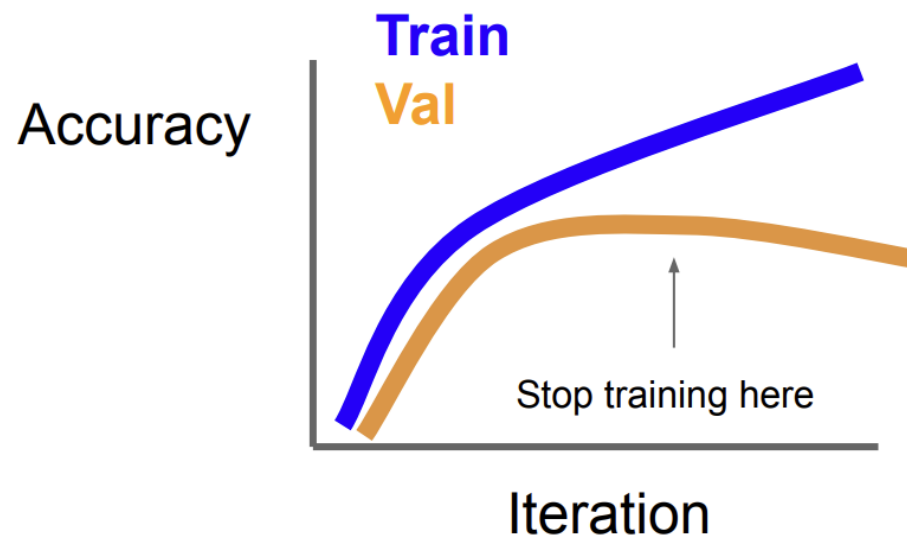
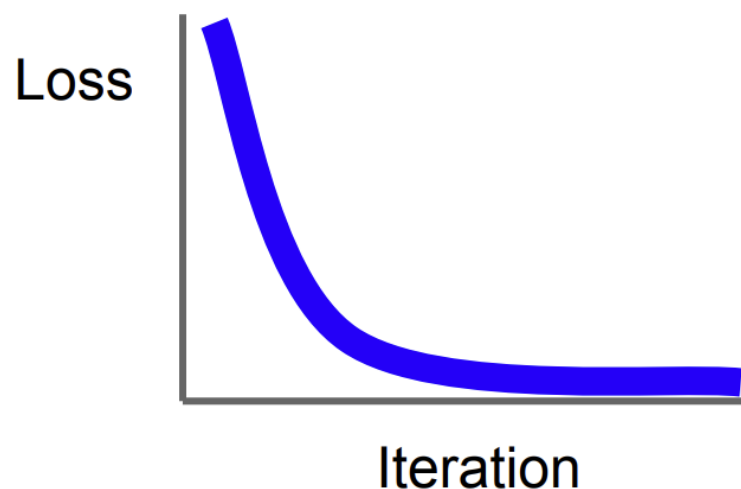
**CNN pretrained
on ImageNet**

Image Captioning: CNN + RNN



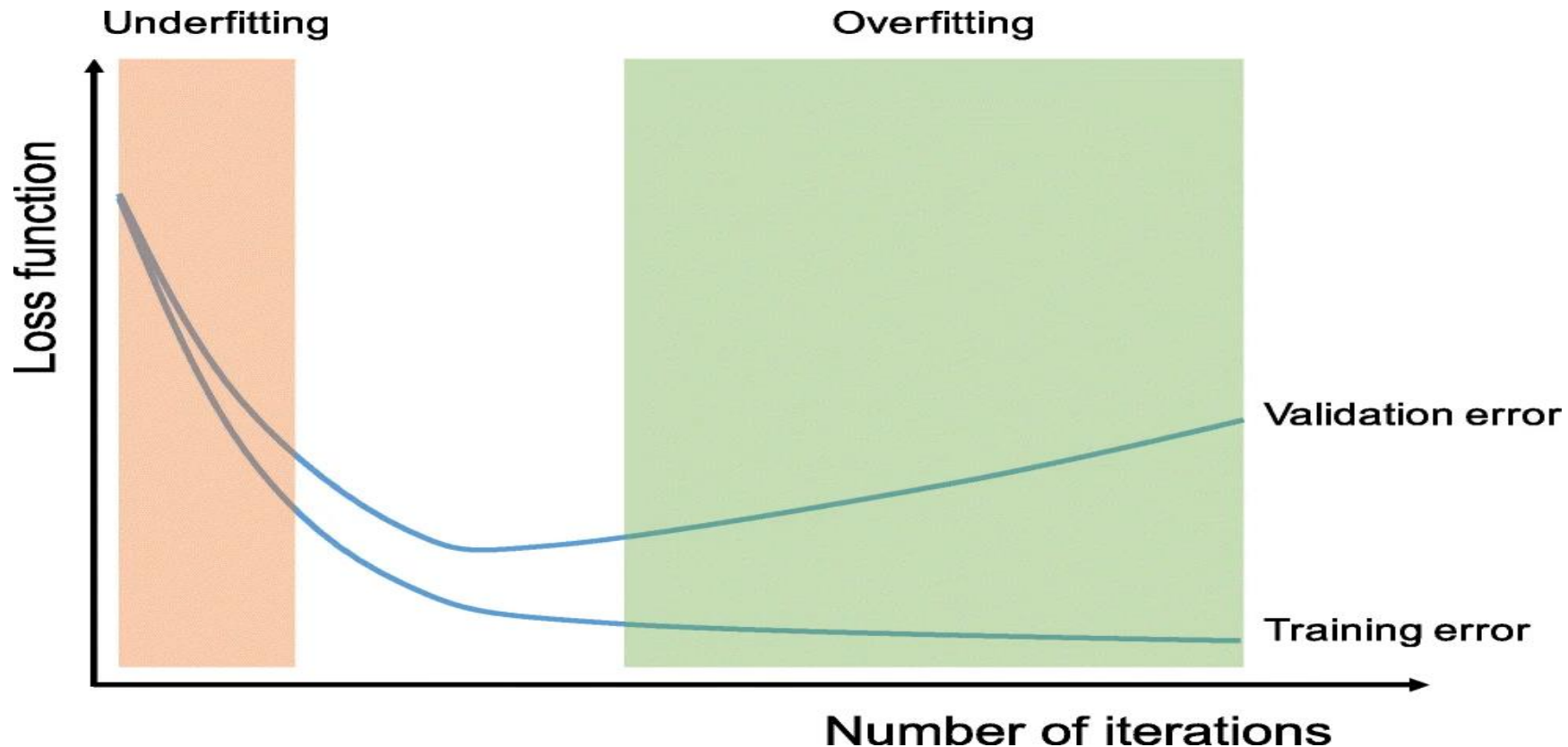
Training vs. Testing Error

- Proper optimizer and training strategy can minimize the loss.
- Small training error is not always corresponding to a small testing/validation error.



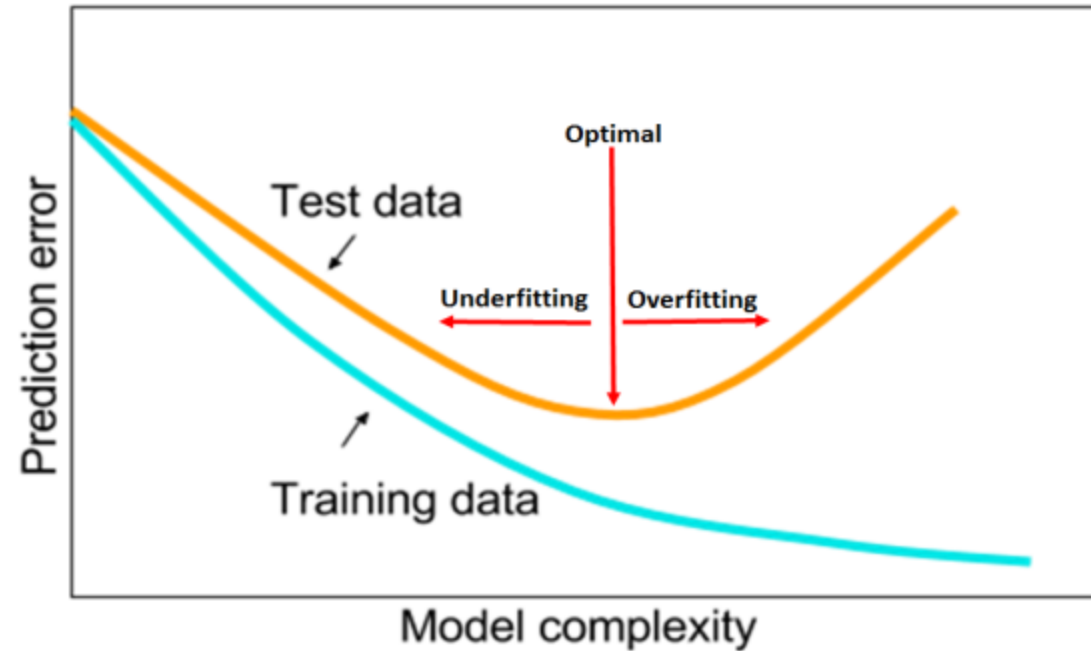
Overfitting and Underfitting

- Monitor the loss on training and validation sets during the training iteration.
- If the model performs poorly on both training and validation sets: Underfitting
- If the model performs well on the training set compared to the validation set: Overfitting



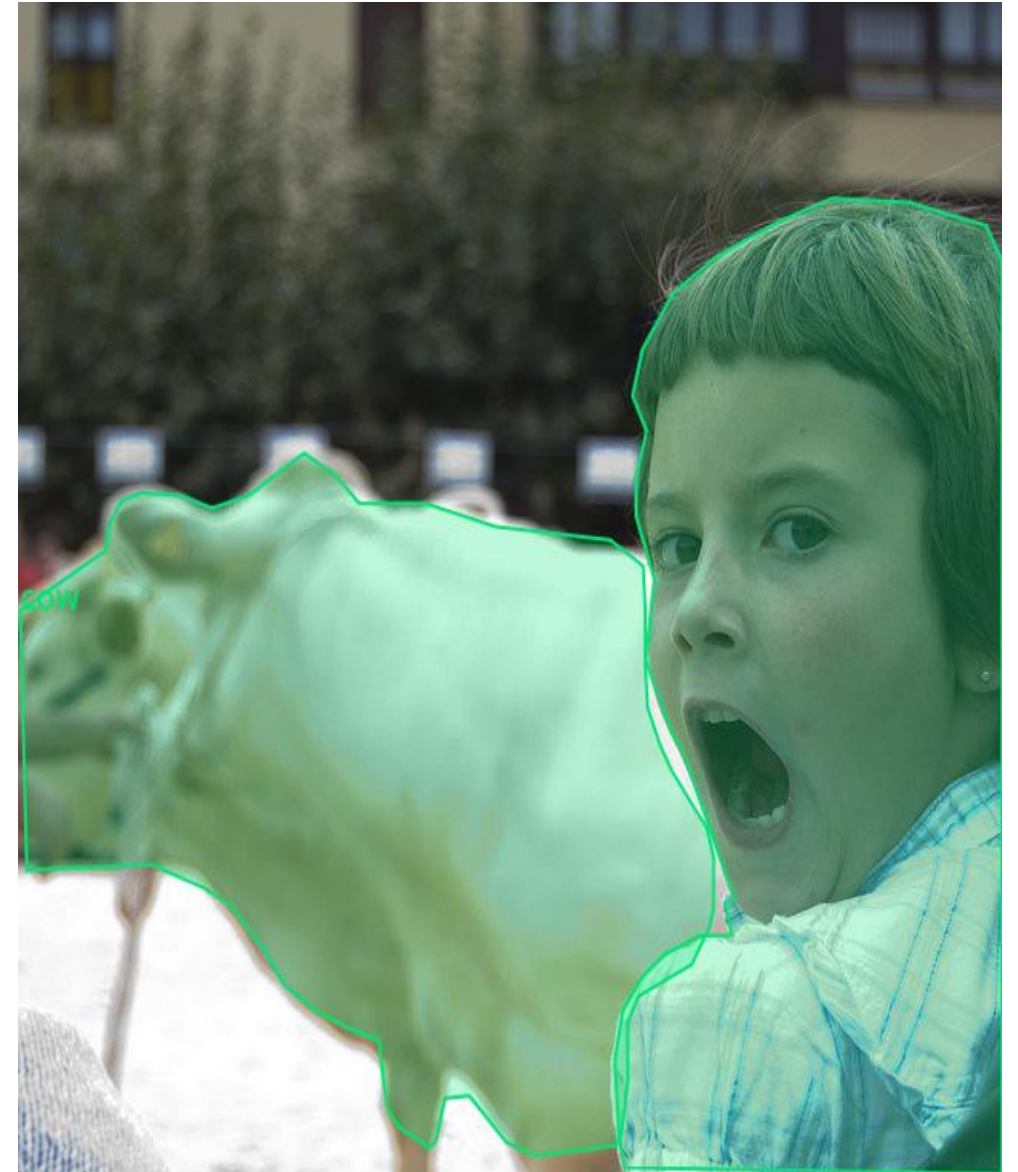
Common methods to mitigate overfitting

- More training data
- Early Stopping
- Data Augmentation
- Regularization (weight decay, dropout)
- Batch normalization



More training data

- Costly
- Time consuming
- Need experts for specialized domains

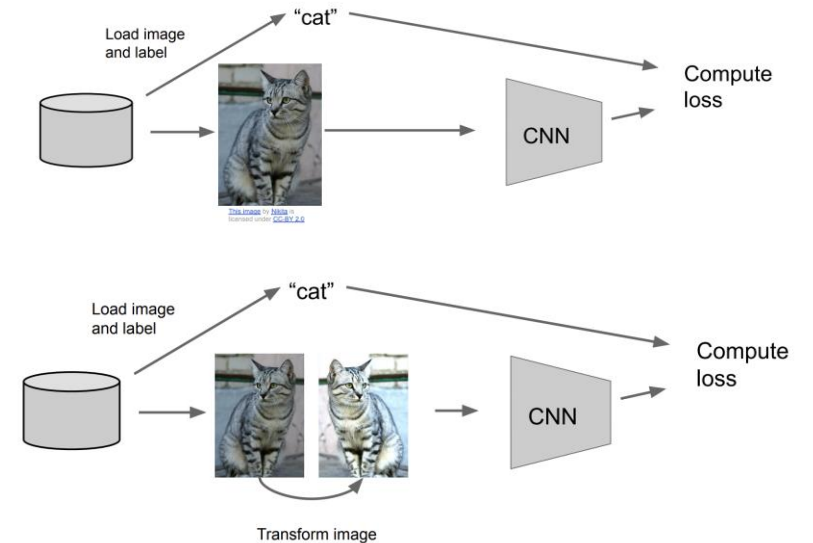


Early Stopping

- Training too little mean model will underfit on the training and testing sets
- Training too much mean model will overfit the training dataset and hence poor performance on test set
- Early Stopping:
 - To stop training at the point when performance on a validation set starts to degrade.
 - Idea is to stop training when generalization error increases
- How to use Early Stopping
 - Monitoring model performance: Using metric to evaluate to monitor performance of the model during training
 - Trigger to stop training:
 - No change in metric over a given number of epochs
 - A decrease in performance observed over a number of epochs
 - Some delay or “patience” is good for early stopping

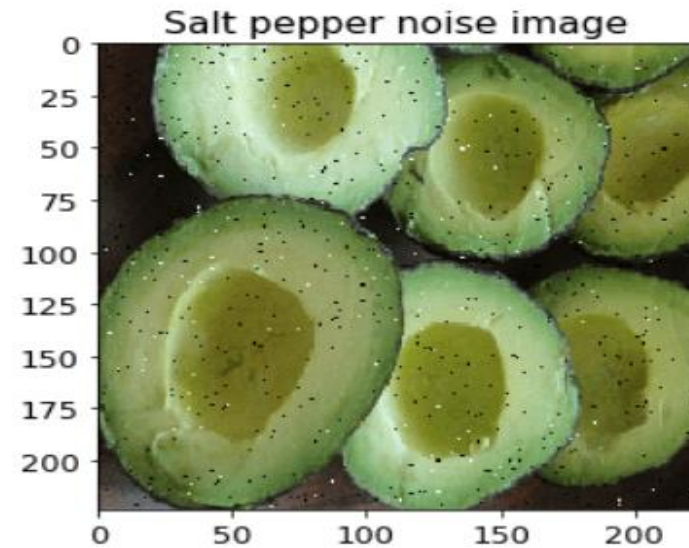
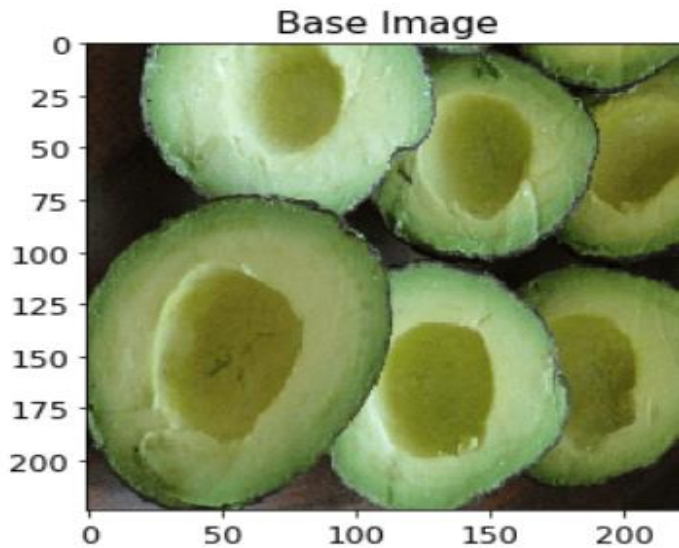
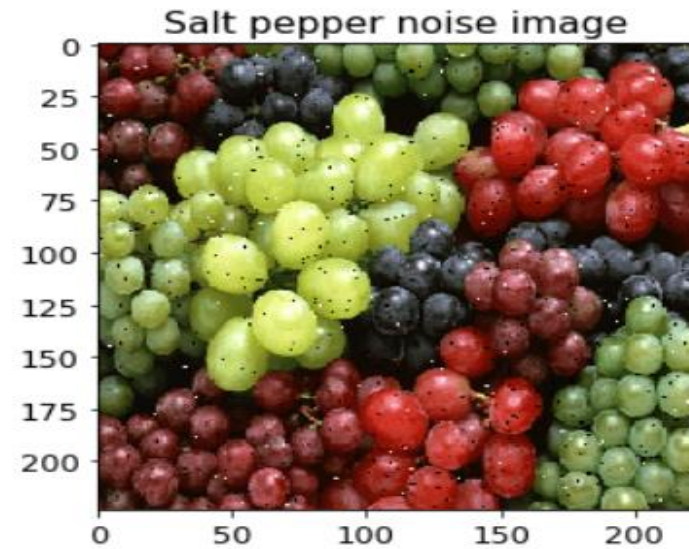
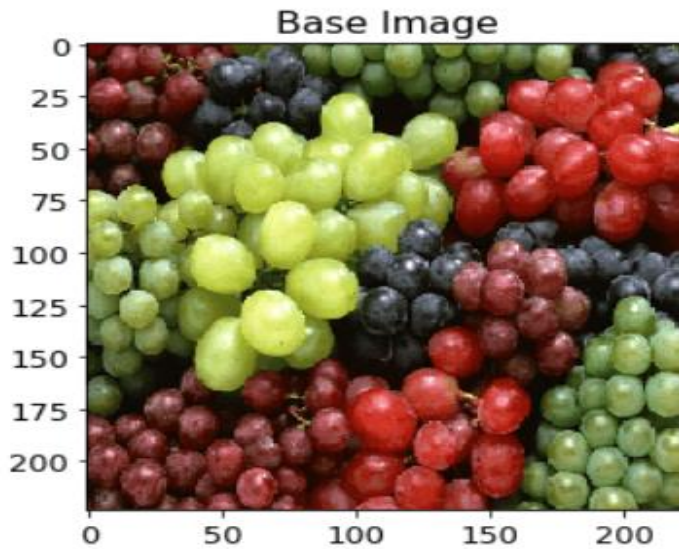
Data Augmentation

- Data augmentation generate different versions of a real dataset artificially to increase its size
- We use data augmentation to handle data scarcity and insufficient data diversity
- Data augmentation helps to increase performance of deep neural networks
- Common augmentation techniques:
 - Adding noise
 - Cropping
 - Flipping
 - Rotation
 - Scaling
 - Translation
 - Brightness
 - Contrast
 - Saturation
 - Generative Adversarial Networks (GANs)



Data Augmentation

➤ Adding noise



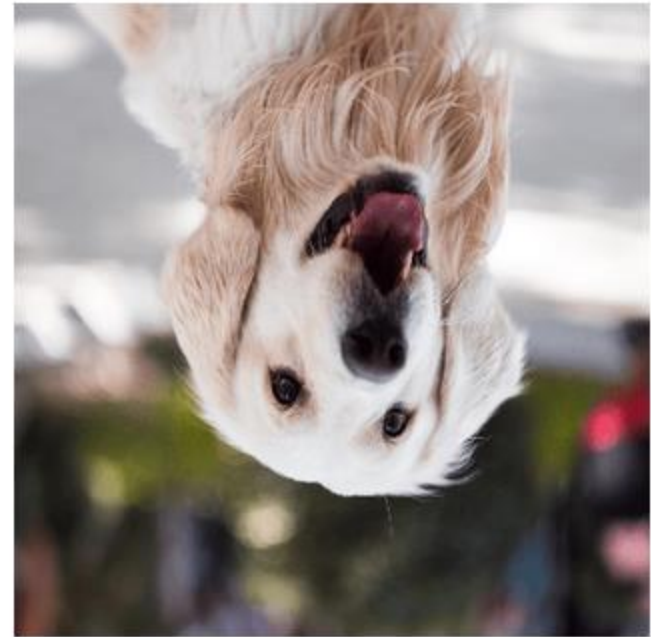
Data Augmentation

➤ Cropping



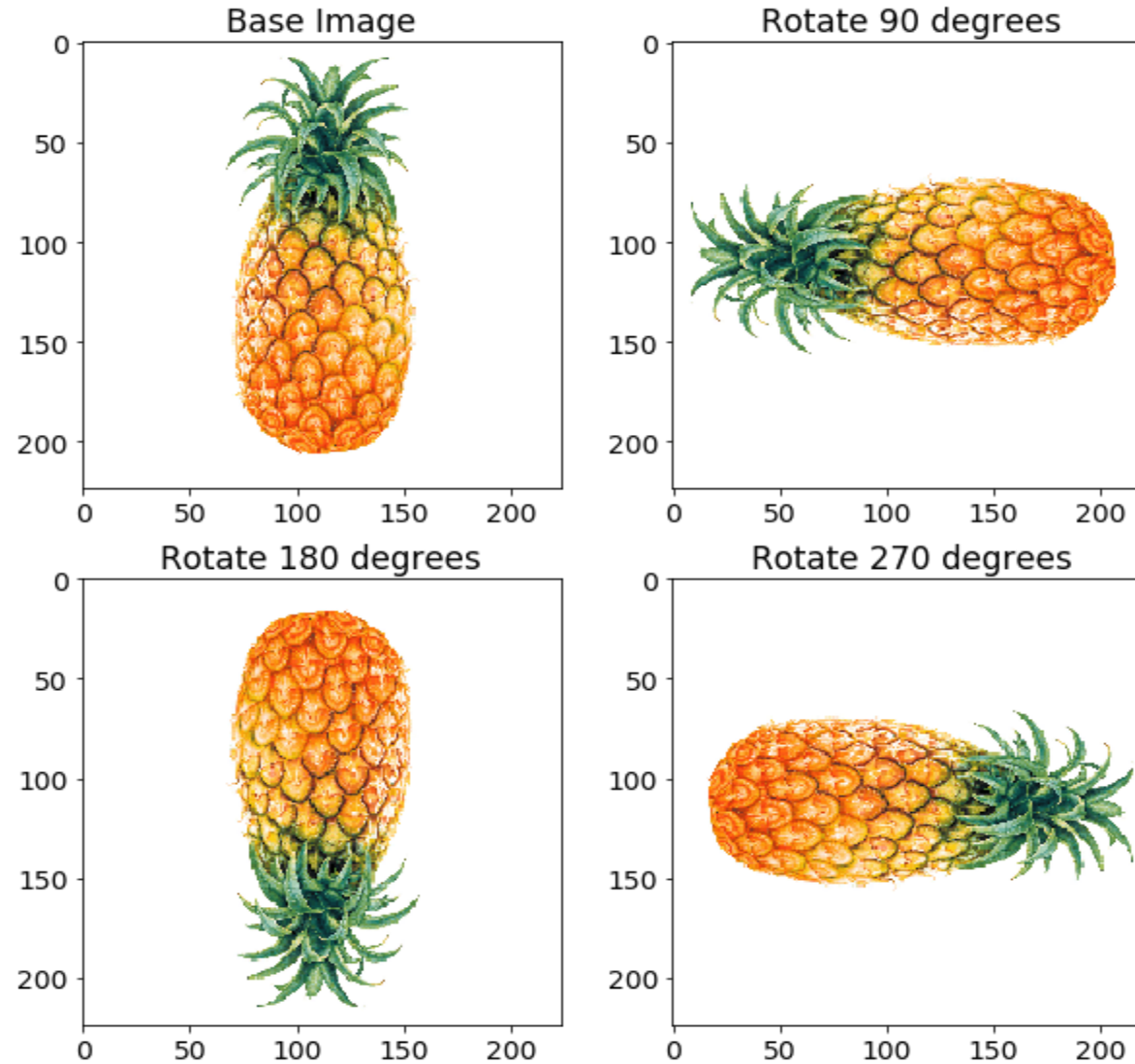
Data Augmentation

➤ Flipping



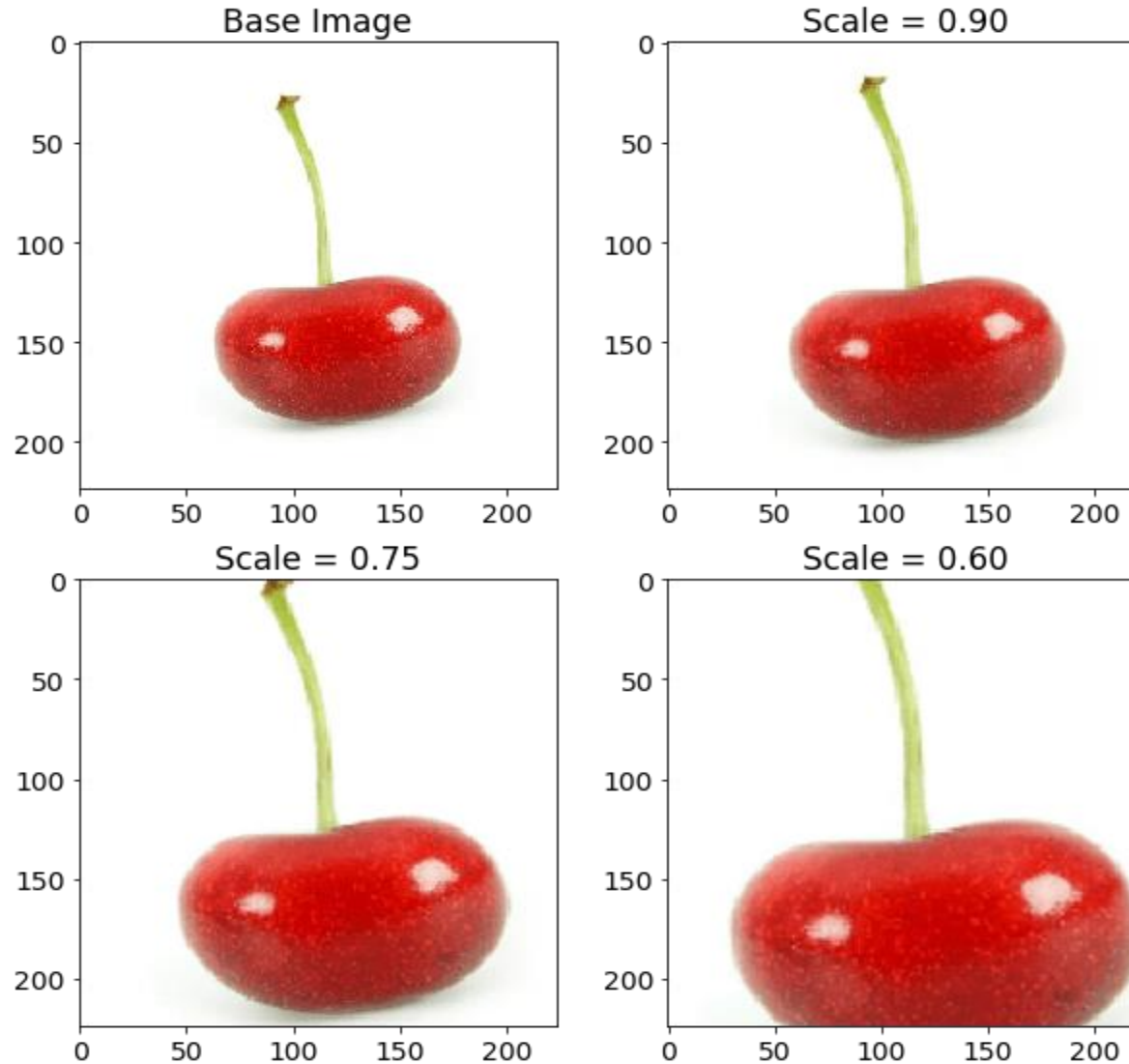
Data Augmentation

➤ Rotation



Data Augmentation

➤ Scaling



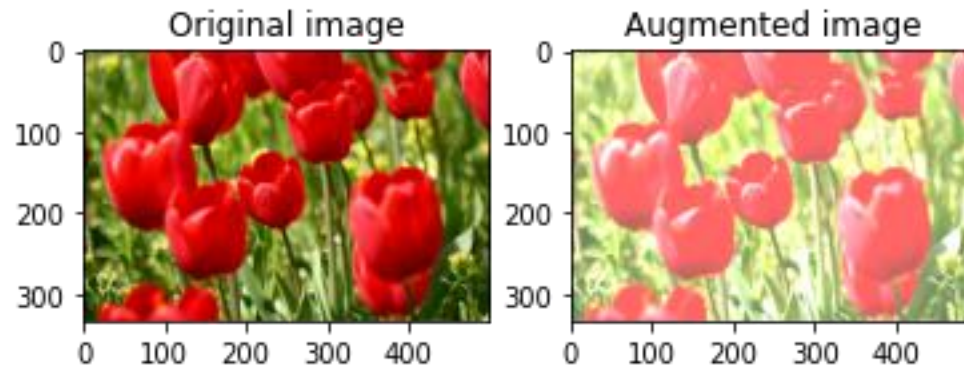
Data Augmentation

➤ Translation



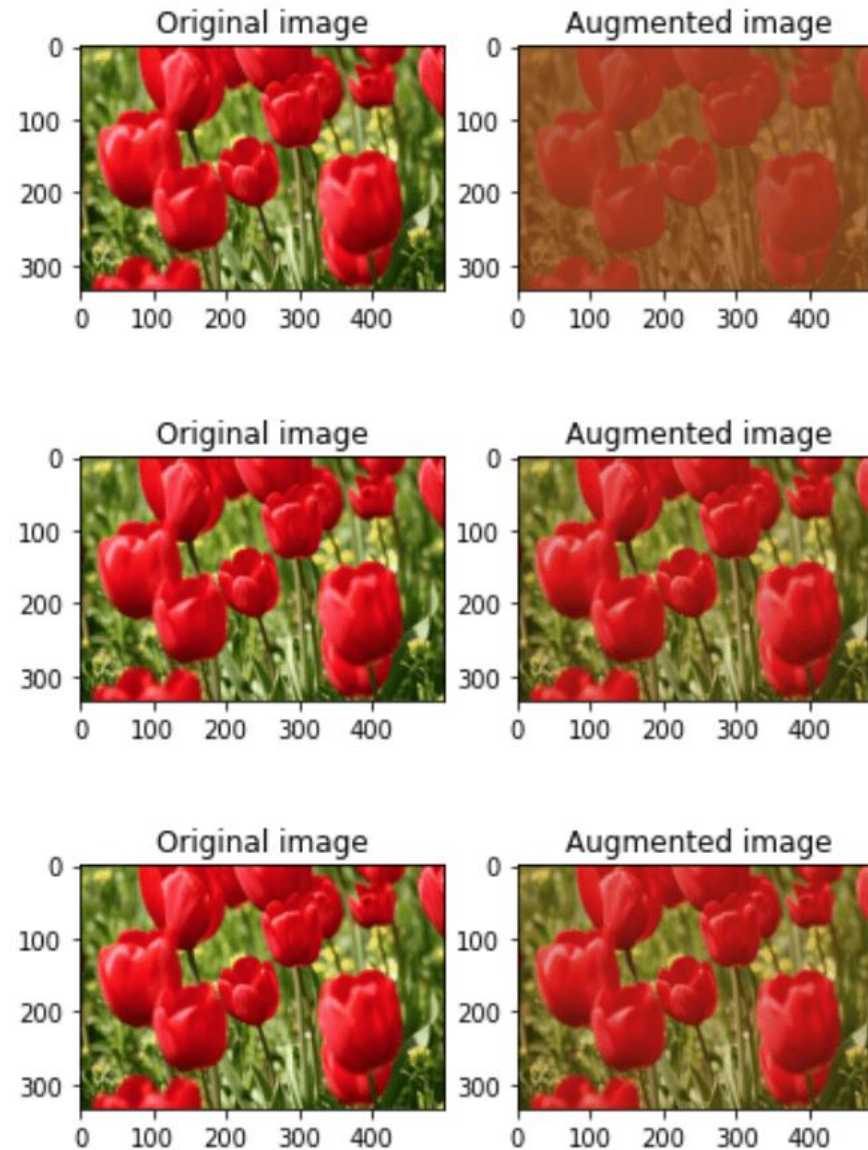
Data Augmentation

➤ Brightness



Data Augmentation

➤ Contrast



Data Augmentation

- Generative Adversarial Networks (GANs) for data augmentation



StyleGAN2 (baseline)



StyleGAN2 + DiffAugment (ours)

Regularization: Weight Decay

- It adds a penalty term to the loss function on the training set to reduce the complexity of the learned model
- Popular choice for weight decay:
 - L1: The L1 penalty aims to minimize the absolute value of the weights

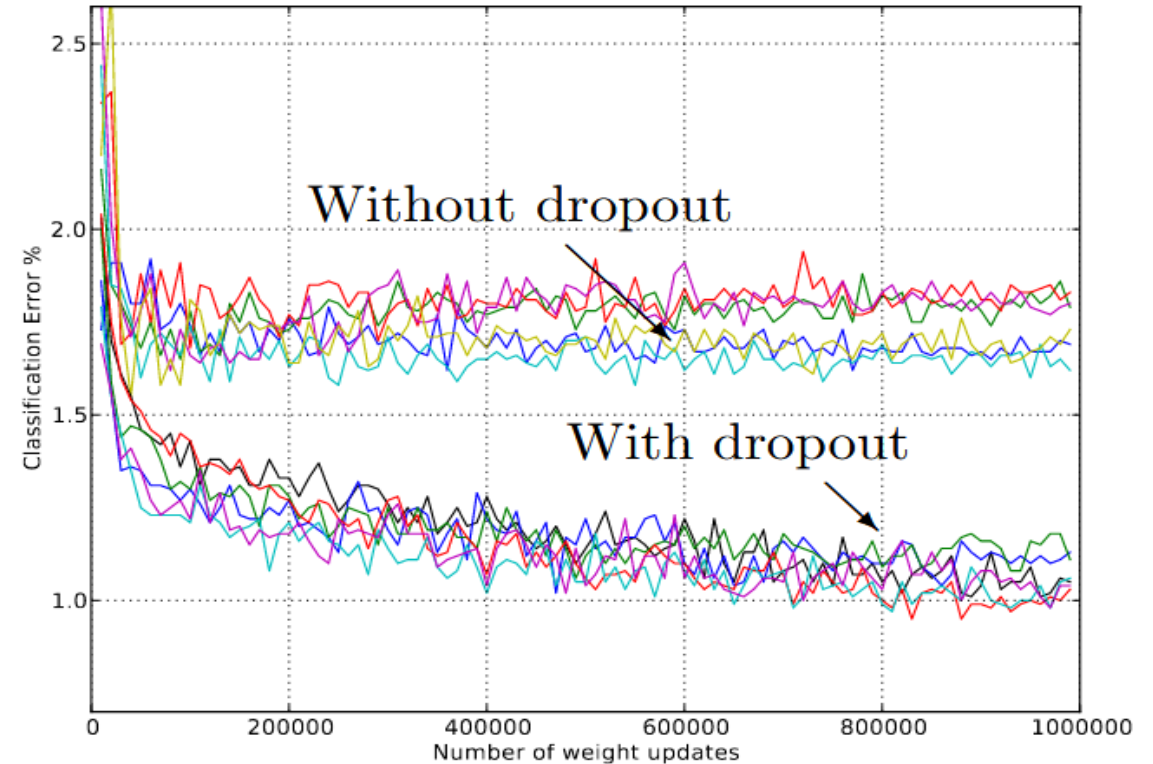
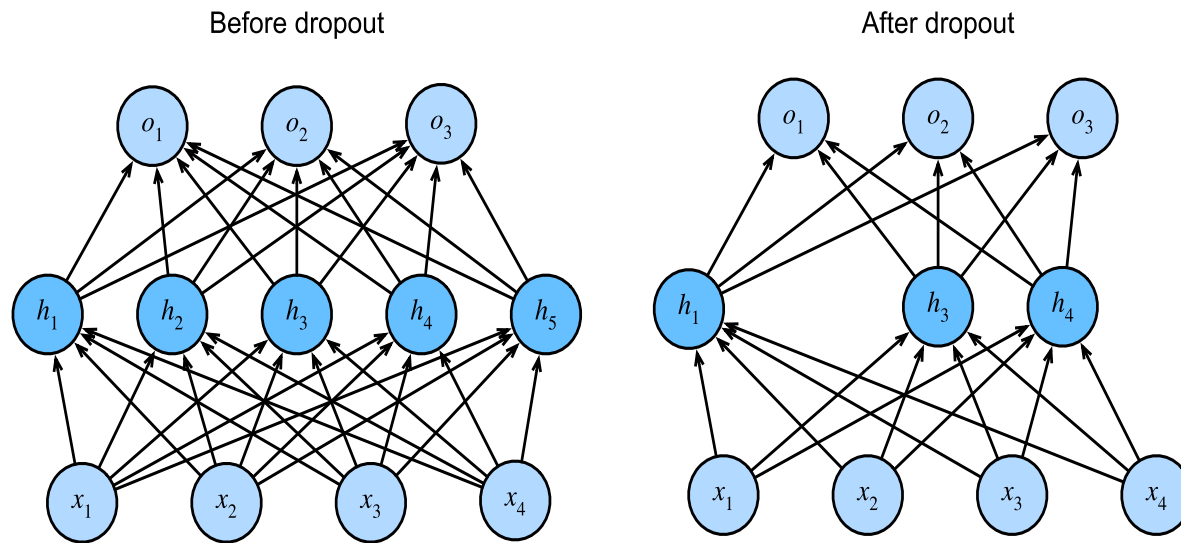
$$L(x, y) \equiv \sum_{i=1}^n (y_i - h_{\theta}(x_i))^2 + \lambda \sum_{i=1}^n |\theta_i|$$

- L2: The L2 penalty aims to minimize the squared magnitude of the weights

$$L(x, y) \equiv \sum_{i=1}^n (y_i - h_{\theta}(x_i))^2 + \lambda \sum_{i=1}^n \theta_i^2$$

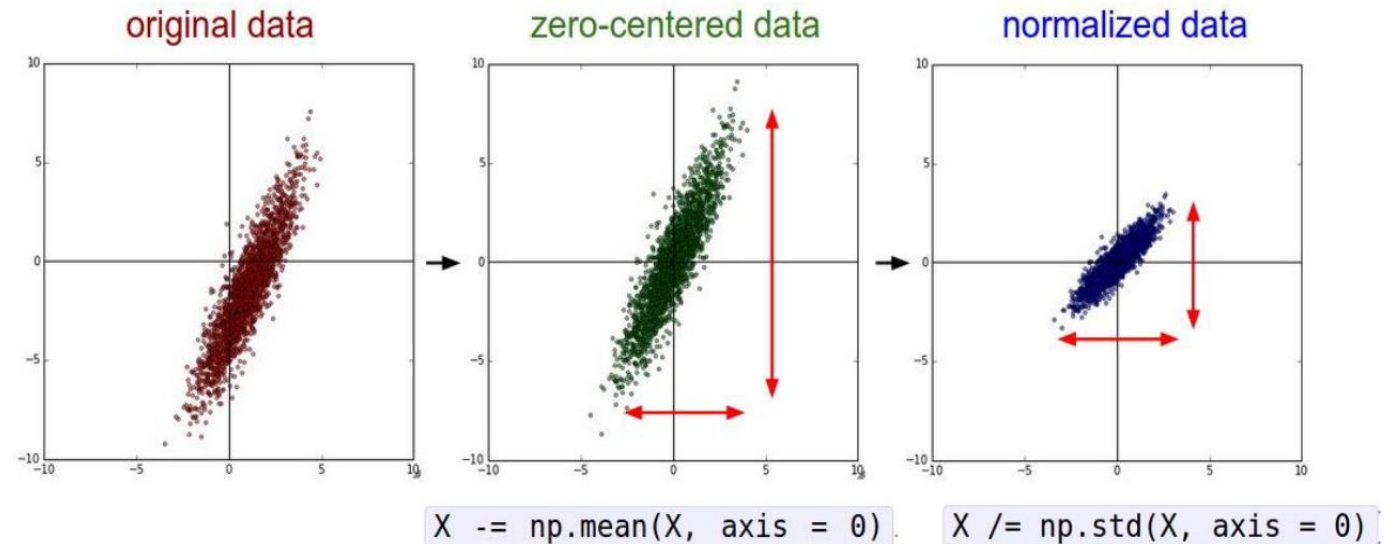
Regularization: Dropout

- L1 and L2 reduce overfitting by modifying the cost function
- Dropout modify the network by randomly dropping neurons from the neural network during training
- Dropout is an efficient way to average many large neural networks



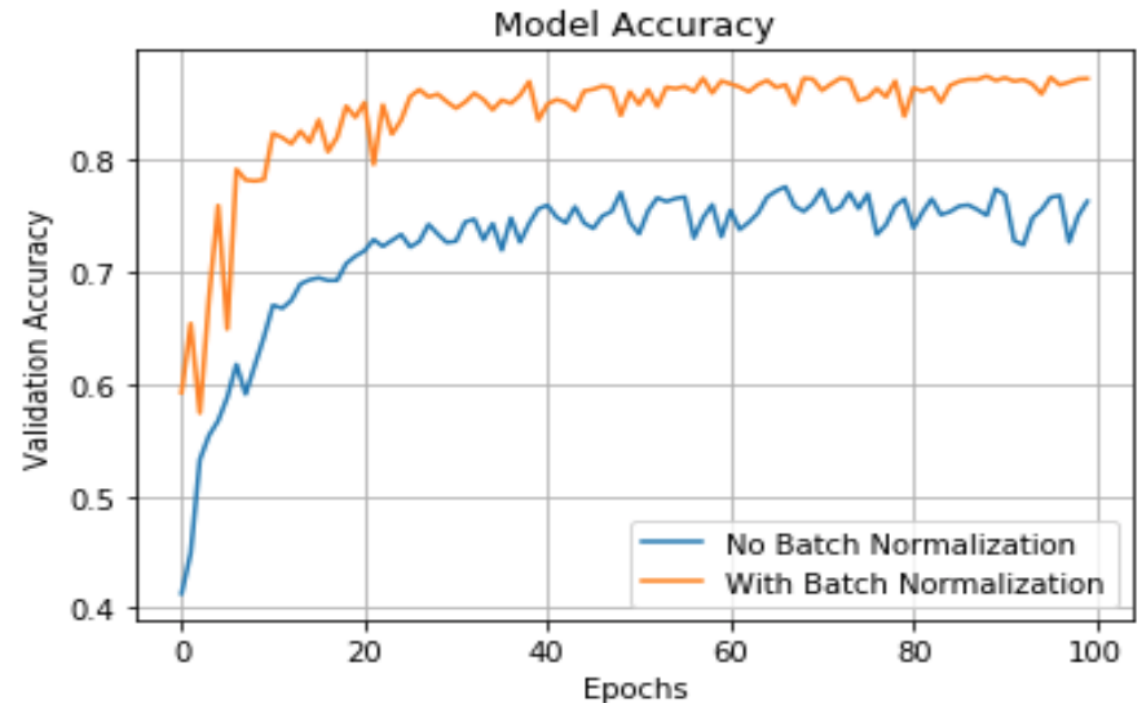
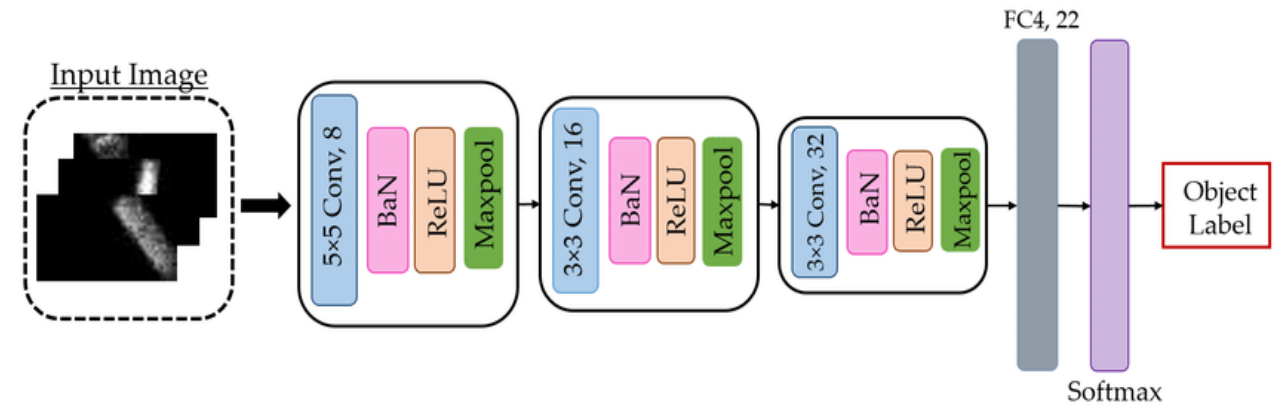
Data Preprocessing

- The pixel values in images must be scaled prior to given as input to deep neural networks for training or evaluation
- Three main types of pixel scaling:
 - **Pixel Normalization**: scale pixel values to the range 0-1
 - **Pixel Centering**: scale pixel values to have a zero mean
 - **Pixel Standardization**: scale pixel values to have a zero mean and unit variance



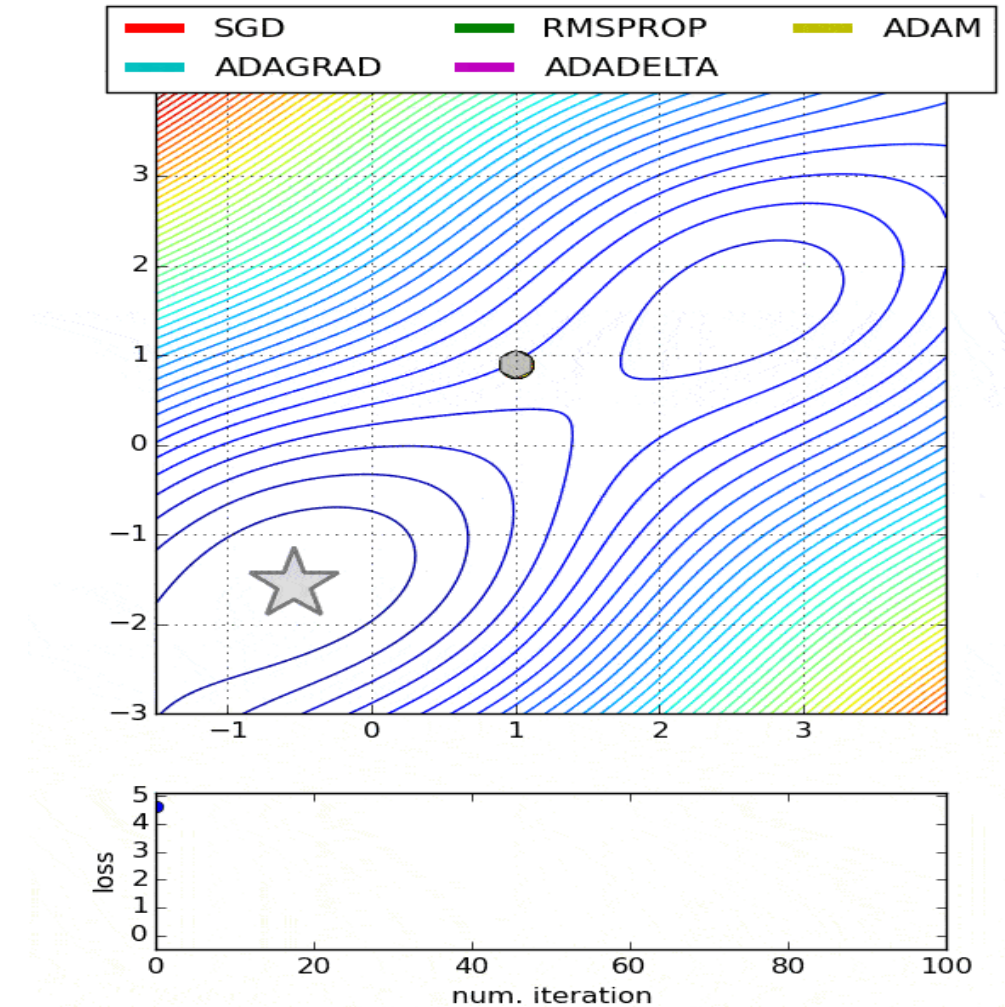
Batch Normalization

- Enables stable training
- Reduces the **internal covariate shift** (ICS)
- Accelerates the training process
- Reduces the dependence of gradients on the scale of the parameters



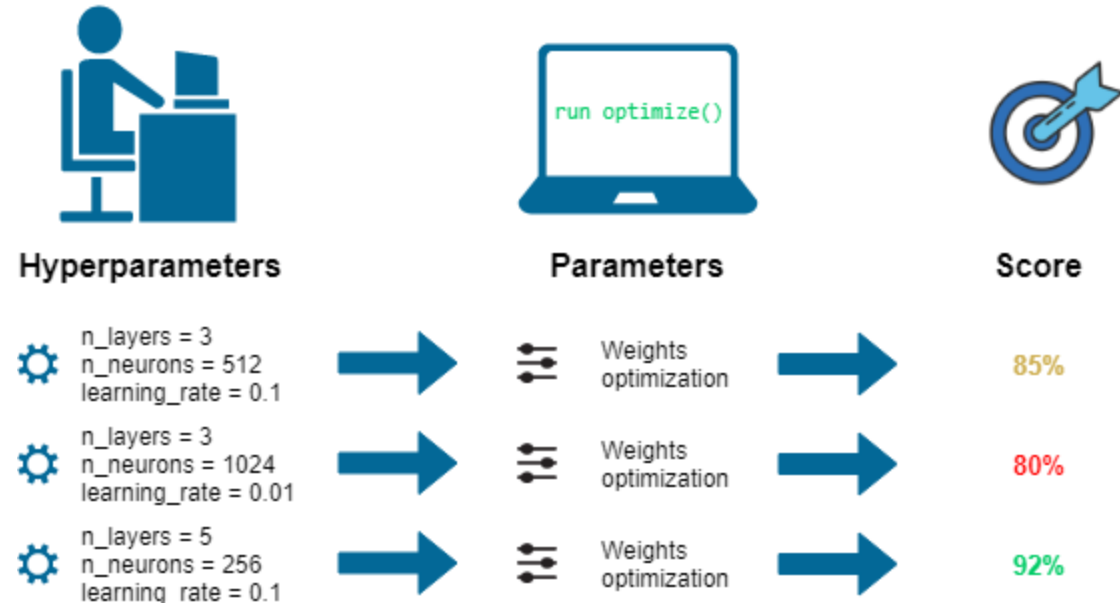
Choice of Optimizers

- Choosing right optimizer helps to update the model parameters and reducing the loss in much less effort
- Most DL frameworks supports various optimizers:
 - Stochastic Gradient Descent (SGD)
 - Momentum
 - Nesterov Accelerated Gradient
 - AdaGrad
 - AdaDelta
 - **Adam**
 - RMSProp



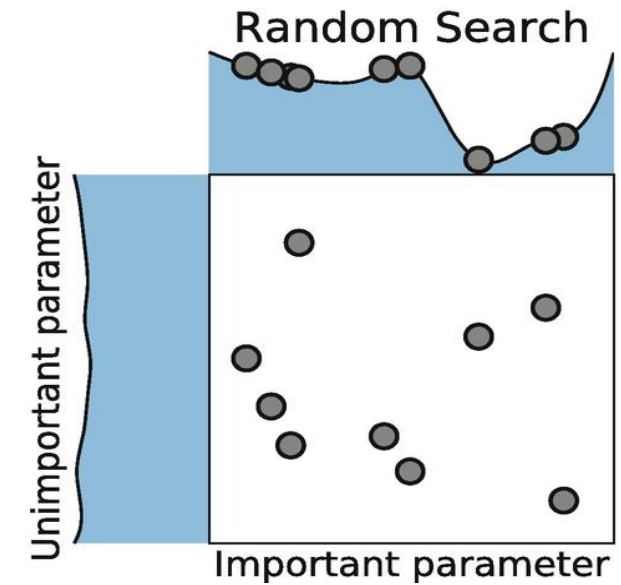
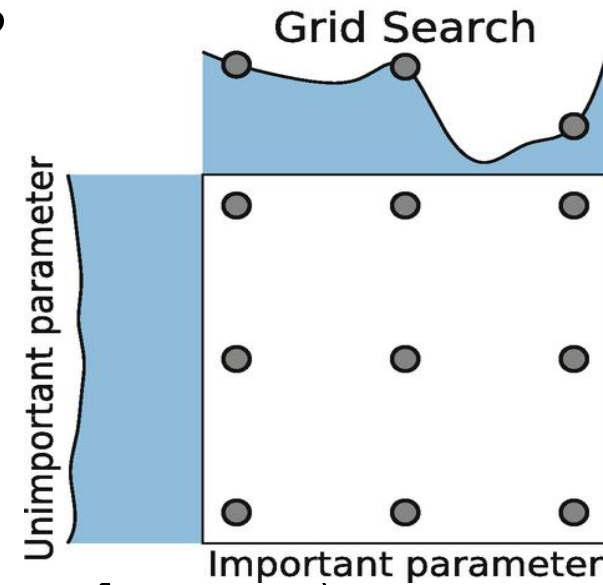
Tuning Hyperparameters

- Hyperparameters are all parameters which can be arbitrarily set by the user before starting training
- Hyperparameters are like knobs or dials of the network (model)
- An optimization problem: We aim to find the right combinations of their values which can help us to find either the minimum (e.g., loss) or the maximum (e.g., accuracy) of a function
- Many hyperparameters to tune:
 - Learning rate
 - No. of epochs
 - Dropout rate
 - Batch size
 - No. of hidden layers and units
 - Activation function
 - Weight initialization
 - ...

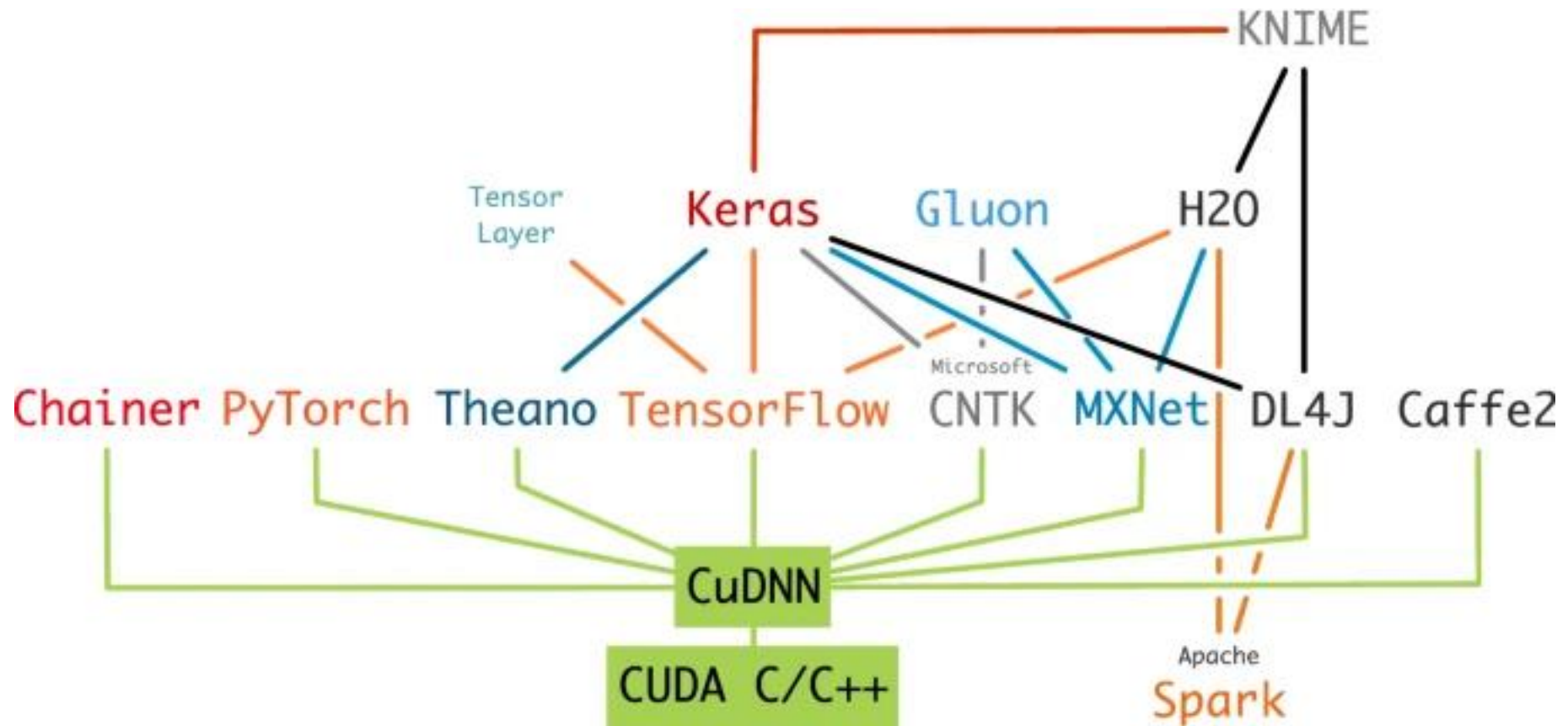


Tuning Hyperparameters strategies

- Random Guess
 - Simply use values from similar work
- Rely on your experience
 - Training DNNs is part art, part science
 - With experience sense of what works and what doesn't
 - Still chances of being incorrect (suboptimal performance)
- Grid Search
 - Set up a grid of hyperparameters and train/test model on each of the possible combinations
- Automated hyperparameter tuning
 - Use of Bayesian optimization and Evolutionary Algorithms
 - Packages: Optuna, Hyperopt



Deep Learning Frameworks



Evaluation Metrics

Classification

Week 4:

Confusion matrix for binary classification

		Prediction	
		P	N
Actual	P	# True Positives (TP)	# False Negatives (FN)
	N	# False Positives (FP)	# True Negatives (TN)

Accuracy

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad \left(\frac{\text{Correct}}{\text{Total}} \right)$$

Classification

Precision / correctness

Fraction of relevant elements among the selected elements

$$\text{Precision} = \frac{TP}{TP + FP} \quad (P)$$

Recall / sensitivity / completeness

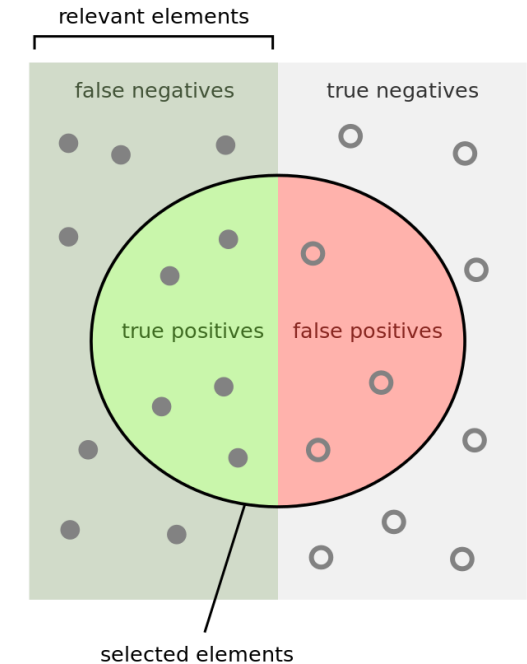
Fraction of selected elements among the relevant elements

$$\text{Recall} = \frac{TP}{TP + FN} \quad (R)$$

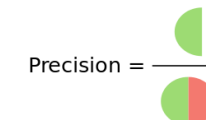
F1 score

Harmonic mean of precision and recall:

$$F1 = \frac{2PR}{P + R}$$

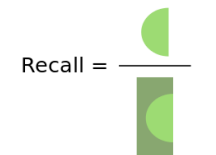


How many selected items are relevant?



$$\text{Precision} = \frac{\text{Green}}{\text{Green} + \text{Red}}$$

How many relevant items are selected?



$$\text{Recall} = \frac{\text{Circle}}{\text{Rectangle}}$$

https://en.wikipedia.org/wiki/Precision_and_recall

Multi-class Metrics

How to calculate metrics in this case?

There are two types of metrics:

1. Calculate metrics for each class, then average each metric over all classes. This is **macro-metrics** calculation. E.g.,

$$TPR = \frac{1}{K} \sum_{i \in K} \frac{TP_i}{TP_i + FN_i}$$

2. Add up values for all classes, creating a new confusion matrix 2 x 2, then calculate all metrics as usual. This is called **micro-metrics** calculation. E.g., over all classes. This is micro-metrics calculation. E.g.,

$$TP = \sum_{i \in K} TP_i$$

		True values		
		Dog	Cat	Panda
Predicted values	Dog	TP	FP	
	Cat	FN	TN	
	Panda			

		<i>gold labels</i>			
		urgent	normal	spam	
<i>system output</i>	urgent	8	10	1	$\text{precision}_u = \frac{8}{8+10+1}$
	normal	5	60	50	$\text{precision}_n = \frac{60}{5+60+50}$
	spam	3	30	200	$\text{precision}_s = \frac{200}{3+30+200}$
		$\text{recall}_u = \frac{8}{8+5+3}$	$\text{recall}_n = \frac{60}{10+60+30}$	$\text{recall}_s = \frac{200}{1+50+200}$	

Class 1: Urgent			Class 2: Normal			Class 3: Spam			Pooled		
	true urgent	true not		true normal	true not		true spam	true not		true yes	true no
system urgent	8	11	system normal	60	55	system spam	200	33	system yes	268	99
system not	8	340	system not	40	212	system not	51	83	system no	99	635

precision = $\frac{8}{8+11} = .42$

precision = $\frac{60}{60+55} = .52$

precision = $\frac{200}{200+33} = .86$

microaverage
precision = $\frac{268}{268+99} = .73$

macroaverage
precision = $\frac{.42+.52+.86}{3} = .60$

Regression/Prediction

Week 4

Root Mean Square Error (RMSE)

Represents the standard deviation of the predicted values from the observed values

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

Mean Absolute Error (MAE)

Represents the average of the absolute differences between the predicted and observed values

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

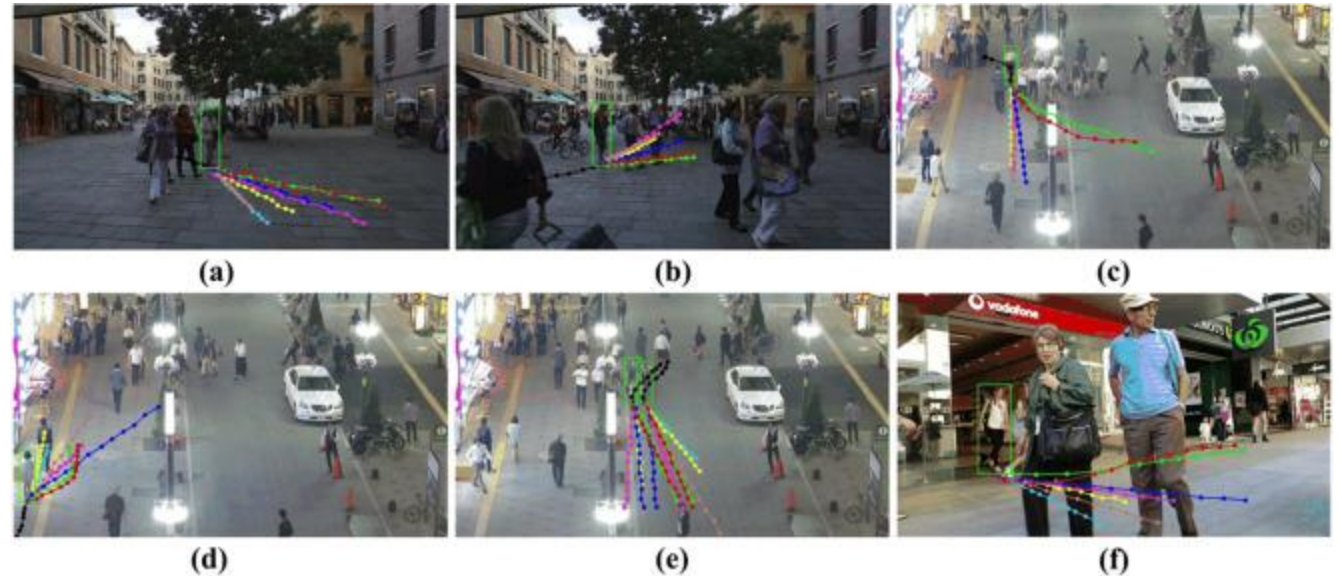
RMSE penalizes big differences between predicted values and observed values more heavily

Smaller values of RMSE and MAE are more desirable

Regression/Prediction Example

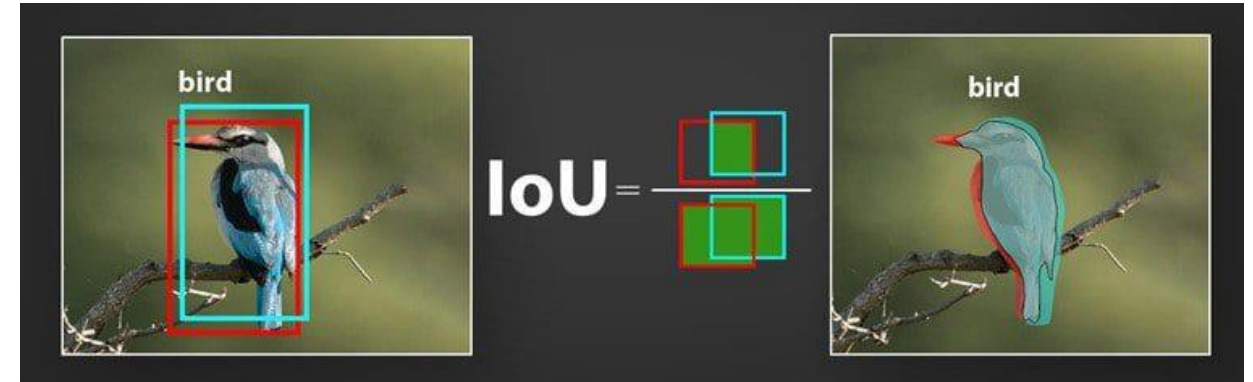
Pedestrian Trajectory Prediction

- ADE: average displacement error (RMSE)
- FDE: final displacement error



More Metrics

- Intersection over Union (IoU)
- Mean Average Precision (mAP)
 - Average Precision (AP): Area under Precision-Recall curve for each class.
 - Mean Average Precision (mAP): Mean of AP values across all classes.
- Jaccard and Dice similarity coefficients (Week 5)
- Mean IoU (mIoU)



	backbone	AP ^{bb}	AP ^{bb} ₅₀	AP ^{bb} ₇₅	AP ^{bb} _S	AP ^{bb} _M	AP ^{bb} _L
Faster R-CNN+++ [19]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [27]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [21]	Inception-ResNet-v2 [41]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [39]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	52.1
Faster R-CNN, RoIAlign	ResNet-101-FPN	37.3	59.6	40.3	19.8	40.2	48.8
Mask R-CNN	ResNet-101-FPN	38.2	60.3	41.7	20.1	41.1	50.2
Mask R-CNN	ResNeXt-101-FPN	39.8	62.3	43.4	22.1	43.2	51.2

Table 3. **Object detection** single-model results (bounding box AP), vs. state-of-the-art on test-dev. Mask R-CNN using ResNet-101-FPN outperforms the base variants of all previous state-of-the-art models (the mask output is ignored in these experiments). The gains of Mask R-CNN over [27] come from using RoIAlign (+1.1 AP^{bb}), multitask training (+0.9 AP^{bb}), and ResNeXt-101 (+1.6 AP^{bb}).

<https://lightning.ai/docs/torchmetrics/stable/>

<https://learnopencv.com/mean-average-precision-map-object-detection-model-evaluation-metric/>

Key takeaways

- Continuous improvement in CNN architectures and heuristics (tips and tricks)
 - always check literature to find state-of-the-art methods
- Training methodology
 - Split data into training (70 %), validation (10 %), and testing (20 %)
 - Take care of data leakage (e.g., multiple samples of same patients should be in same set)
 - Check distribution of classes, work on balanced datasets (ideally)
 - Tune hyperparameters on validation set. Save best model and do inference on test set (once)
 - Don't use off-the-shelf model blindly. Do ablation studies to know its working
- Data augmentation techniques are not standardized
 - Get input from experts to know what data augmentations make sense in the domain
 - For e.g., in chest X-rays we don't want vertical flipping
- Results
 - Use multiple metrics rather a single metric to report results (often they are complementary)
 - Show both qualitative and quantitative results (e.g., image segmentation)